

# RTL Design Sherpa

# ALU-INFER Micro-Architecture Specification 0.50

**October 18, 2026**

# Table of Contents

1 Alu Infer Mas Index....12

2 ALU-INFER — Cell Inference ALU Product Requirements Document....13

2.1 1. Identity and scope....14

2.2 2. Element model....15

2.3 3. Function set (v1)....16

2.3.1 Function-set roadmap (owner ruling 2026-10-19)....17

2.4 4. Interface contract....19

2.5 5. Control interface — gatling issues the ALU; in-band everything....23

2.6 6. Verification hooks....24

2.7 7. Deferred and growth....25

2.8 8. Open questions....26

2.9 9. Dependencies....27

2.10 10. References....28

3 GATLING PRD — Tetrad Compute Engine....29

3.1 1. Purpose....30

3.2 2. Non-goals....31

3.3 3. ISA — RV32I + Xcell....32

3.3.1 3.1 Xcell encoding (custom-0, opcode 0x0B)....32

3.3.2 3.2 Assembly mnemonics....34

3.4 4. Barrel microarchitecture....35

3.4.1 4.1 The model....35

3.4.2 4.2 Structural hazards (all resolved by rotation, by construction)....35

3.4.3 4.3 Branch behavior....35
3.4.4 4.4 Halt/trap....35
3.5 5. Contract fit (cell PRD 0.1, unchanged interfaces)....36
3.6 6. Verification....37
3.7 7. Open items (owner decisions needed)....38
3.8 8. Deliverables & sequencing....39
3.9 9. History....40
4 Cell — Compute Engine IP Product Requirements Document....41
4.1 1. Identity and scope....42
4.2 2. Architecture....43
4.2.1 Figure 1: Cell top-level block plan (house style — three flows: programming down the center, operand fill on the left, results and credits back up the right)....43
4.3 3. Dataflow....46
4.4 4. ALU and execution FSM....47
4.5 5. Packet interface....48
4.6 6. Credit behavior....49
4.6.1 6.1 Boot advertisement....49
4.6.2 6.2 Consumed-line credit returns....49
4.6.3 6.3 Straps on the result path....49
4.7 7. Config registers....51
4.8 8. Status and observability....54
4.9 9. Verification hooks....55
4.10 10. Dependencies....56

4.11 11. Deferred ..... 57
4.12 12. Open questions ..... 58
4.13 References ..... 59
5 alu-infer — Cell Inference ALU (definition) ..... 60
6 rtl-math module index ..... 61
6.1 Overview ..... 62
6.2 Module categories ..... 63
6.2.1 Basic arithmetic ..... 63
6.2.2 Advanced adders ..... 63
6.2.3 Division ..... 64
6.2.4 Subtraction ..... 64
6.2.5 Multiplication ..... 64
6.2.6 BF16 floating-point arithmetic ..... 65
6.2.7 FP16 (half-precision) floating-point ..... 66
6.2.8 FP32 (single-precision) floating-point ..... 66
6.2.9 FP8 (8-bit floating-point) ..... 66
6.2.10 IEEE 754-2008 compliant arithmetic ..... 66
6.2.11 Compressors and prefix cells ..... 66
6.3 Related Modules ..... 68
6.4 Navigation ..... 69
7 Document Information ..... 70
7.1 Overview ..... 70
7.1.1 ALU-INFER Micro-Architecture Specification ..... 70
7.1.2 Document Purpose ..... 70

7.1.3 Intended Audience....71
7.2 Design Notes....72
7.2.1 Revision History....72
7.3 References....75
7.3.1 Related Documents....75
8 Design Philosophy and Datapath Organization....76
8.1 What This Block Is....76
8.2 The Dataflow in One Paragraph....77
8.2.1 Figure 1.1: ALU-INFER datapath overview....77
8.3 Element Formats....78
8.4 Relationship to Gatling and the Cell....79
8.5 Status of This Specification....80
9 alu_infer_top (module tree)....81
9.1 Purpose....81
9.2 Module Tree....82
9.2.1 Figure 2.1: ALU-INFER full block diagram....83
9.3 Parameters....84
9.4 Clock and Reset....86
10 The Function Rows....87
10.1 Opcode Map....88
10.2 Elementwise Rows per Format....90
10.3 Per-Row Notes....91
10.4 Activation and Scaling Rows....92
10.5 What Every Row Shares....93

11 The Accumulator Rows: MAC and REDUCE....94
11.1 Why a Wide Accumulator....94
11.2 Structure....95
11.3 Why the Loop Stays Fused....96
11.4 acc_ctrl....97
11.5 Presentation (decision M7)....98
11.6 REDUCE and the Cross-Lane Fold....99
11.7 Isolation Invariants....100
12 Decode and Result Ports....101
12.1 Decode (alu_infer_decode)....101
12.2 The Four Result Ports....102
12.3 The Tag FIFO and Arrival Join (alu_infer_pipe)....103
13 The PWL LUT Activation....104
13.1 Segments....105
13.2 Evaluation....106
13.3 Loading In-Band....107
13.4 The Table Builder....108
14 Format Conversions....109
14.1 Lane Packing....110
14.2 Pairs....111
14.3 ALTFMT (bf16 only)....112
15 The Issue Seam....113
15.1 Signal Contract....114
15.2 Handshake Rules....116

15.2.1 Waveform 3.1: Elastic handshake — natural latency, output stall, ready stays high....117
15.2.2 Waveform 3.2: Back-to-back issue — mixed latencies, in-order results....117
15.3 Boundary Notes....118
16 The Compute Cycle, per Opcode Class....119
16.1 Unary Rows (copy, relu, activations, PWL, const)....120
16.2 Binary Rows (add, sub, mul, min/max, compare)....121
16.3 Conversions....122
16.4 MAC and REDUCE (the chains that matter)....123
16.4.1 Waveform 4.1: MAC chain — clear, three accumulates, drain....123
16.5 Overlapping Invocations....124
17 Verification Hooks and Observability....125
17.1 The Golden Models: the Library's Own....126
17.2 Scenarios....127
17.3 Build Configurations....128
17.4 Coverage Matrix....129
17.5 Observability....130
18 Parameters, Encodings, and Open Items....131
18.1 Parameters....131
18.2 func_sel Encoding....132
18.3 acc_ctrl Encoding....133
18.4 threshold_imm Uses....134
18.5 Filelists....135

18.6 Open Items Register..... 136

# List of Figures

Figure 1: Cell top-level block plan (house style — three flows: programming down the center, operand fill on the left, results and credits back up the right).....43
Figure 1.1: ALU-INFER datapath overview.....77
Figure 2.1: ALU-INFER full block diagram.....83

# List of Tables

Table 1: Document information....70
Table 2: Revision history....72
Table 3: Table 1.1: element formats....78
Table 4: Table 2.1: alu_infer_top parameters....84
Table 5: Table 2.2: the function rows....87
Table 6: Table 2.3: elementwise rows....89
Table 7: Table 2.4: accumulate operation per format....94
Table 8: Table 2.5: accumulator actions....96
Table 9: Table 2.6: result ports....101
Table 10: Table 2.7: modeled PWL accuracy (relative error only where |f| > 2^-6) ....107
Table 11: Table 2.8: CVT pairs....110
Table 12: Table 2.9: ALTFMT sub-ops....111
Table 13: Table 3.1: issue-seam signals....113
Table 14: Table 5.1: test scenarios....126
Table 15: Table 5.2: build configurations....127
Table 16: Table 5.3: coverage matrix....128
Table 17: Table 6.1: func_sel fields....131
Table 18: Table 6.2: acc_ctrl encoding (accepted 2026-10-17, “works for now”)...132
Table 19: Table 6.3: immediate uses....133
Table 20: Table 6.4: open items M1–M8....135

# List of Waveforms

Waveform 3.1: Elastic handshake — natural latency, output stall, ready stays high .....116
Waveform 3.2: Back-to-back issue — mixed latencies, in-order results.....116
Waveform 4.1: MAC chain — clear, three accumulates, drain.....122

# 1 Alu Infer Mas Index

**Generated:** 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

**← Back to compute-eng-ip**

# 2 ALU-INFER — Cell Inference ALU Product Requirements Document

**Version:** 0.50 (2026-10-18 — **revs pinned at 0.50 per owner, "for now,"** while the ALU heads to the board; this rev accumulates every ruling to date). Accumulated since the v0.3 base: **tag-based elastic alignment** (§4 invariant 3) — each loaded invocation takes an 8-bit tag, common to all ops that must execute in parallel; a FIFO tracks tags in flight; an op asserts its valid only when every op of its tag group is ready; tags stay inside the block and the seam keeps its two valid/ready pairs only; enables the §7 growth item (deep-pipelining the multiplier / FMA rows with the adder's `PIPE_STAGE_*` pattern). **Tetrad-engine deferral / gatling ruling 2026-10-19** (§5) — the per-tetrad core is defined after the cell is fully defined and working; the owner-authored gatling-rv32itetrad barrel-core PRD is the **selected sequencer** for its tetrad (kestrel-class stays a `CORE_FLAVOR` candidate). **Port-width correction** — operand ports stay `LINE_W` each (one line per side, both captured in the same clock); the earlier "widening" was a line-vs-lane mis-count. **Owner accept-and-defer** — M7 and `acc_ctrl` accepted as drafted; per-row latency work deferred to FPGA bring-up (M8). **Elastic-pipeline interface** — valid/ready at both ends, backpressure freezes the alignment tracking, II=1 while draining, no padded stages on quick rows, in-order results. **Base decisions** — bf16 element model on the rtl/math `math_bf16_*` library, fp32 MAC accumulate, one RV32I core per tetrad with in-band everything. **Implementation (2026-10-20, revs still pinned):** every row of §3 — the v1 table, the function-set roadmap (reduce, PWL/LUT, const), the format roadmap (int8, fp32, plus fp8 e4m3 from §7) and the growth list — is built and verified, except what this PRD rules out (a true divider, the B4–B32 companion mode, the CGRA replacement). The issuer is gatling (§5). The MAS is the as-built record. **Status:** Pre-HAS PRD, and the **normative definition of the cell's ALU block**: the RTL implementation lives in this component (`alu-infer/rtl/`, unit tests in `alu-infer/dv/tests/`; moved from `cell/rtl/` 2026-10-20) and the **cell** component integrates it. Where this document and the cell PRD touch the ALU, this one governs (the cell PRD was reconciled to it 2026-10-17; see §2).

---

## 2.1 1. Identity and scope

**alu-infer** is the compute engine of the **cell**: the function array the tetrad's **gatling** core issues (§5) to turn input-memory lines into result-memory lines. It is an *inference* ALU — the arithmetic a neural-network layer needs (add, multiply, fused accumulate, min/max, threshold, ReLU-family activations), not a general-purpose ISA datapath.

**Component base (owner direction 2026-10-17): the implementation is built mostly from the `math_bf16_*` family of `rtl/math/`** — the repo's existing brain-float arithmetic library. Nearly every v1 function row has a block there (adder, multiplier, FMA, comparator, min/max, ReLU, and the rest of the activation set), so the ALU is the math library's inference consumer the same way the falcon suite's garuda rung is its IEEE-fp32 consumer. Rows without a bf16 block (copy, and the compare *mask* packing) are wires and glue.

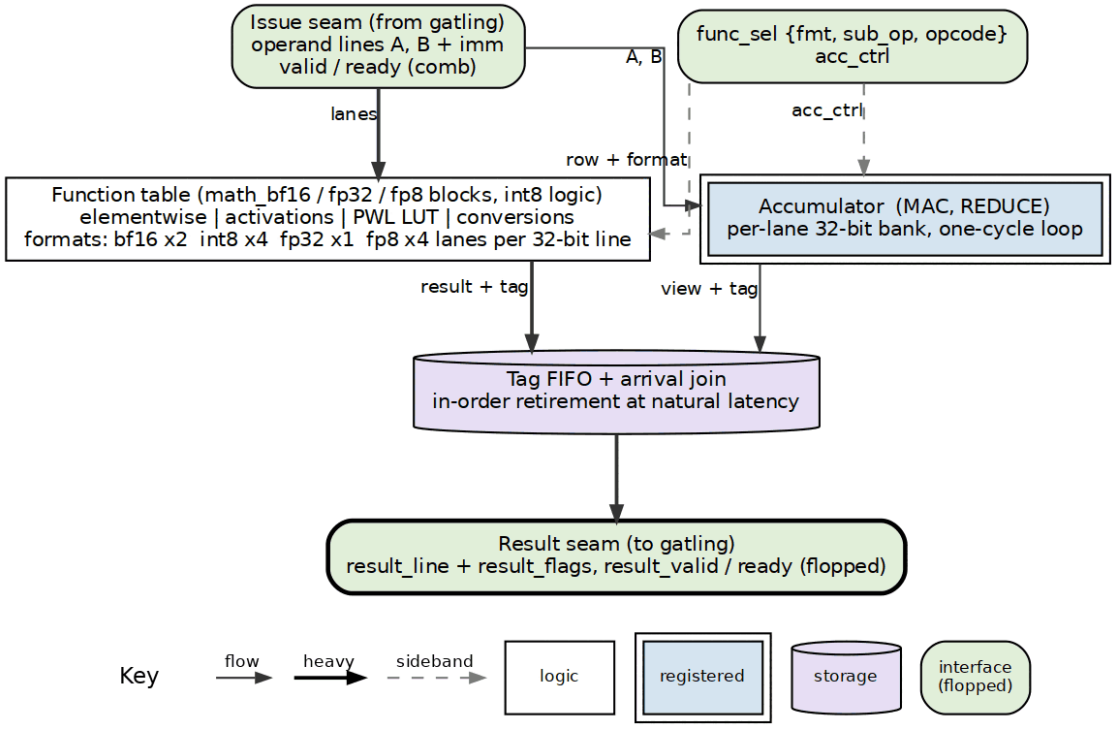
Placement (owner decision 2026-10-17, updated 2026-10-20): this directory holds the product definition, verification doctrine, the RTL (`rtl/`, SystemVerilog), and its cocotb unit tests (`dv/tests/`) — organized like the repo's other component areas (the dma-ip/stream pattern: flat RTL with per-block filelists and a lint master, a four-line area Makefile, and a discoverable `dv/tests` suite). The `cell/` component integrates the block as the cell's "ALU" element (gatling replaced the execution FSM as its driver, owner ruling 2026-10-19): the cell filelists include this block's filelist, and cell-level tests will verify it with the memories, packetizer, and credit logic it serves. The block boundaries in the cell PRD were chosen so a future CGRA array replaces this ALU without disturbing the network, credit, or observation contracts (cell PRD §11); this PRD's interface contract (§4) is written to that same seam.

Out of scope: the issuer's sequencing policy (gatling's), the memories, the packetizer, credit/grant logic, and the delta node (all cell PRD territory).

## 2.2 2. Element model

Owner direction 2026-10-17: a **32-bit line is read as 2×bf16 elements** — two 16-bit brain-float lanes — and the function array is built from `math_bf16_*` blocks. bf16 keeps fp32's 8-bit exponent and truncates the mantissa to 7 bits, which is why the library's inference blocks (and the quantization literature behind them) chose it: full fp32 range at half the storage. The packed-line structure of cell PRD §4 is retained: the configured function applies to every element in the line **in parallel** (2 lanes per 32-bit line), and wider lines (cell PRD §12) scale the lane count, never the element width.

**Supersession, reconciled.** Cell PRD §4's element-width note (owner, 2026-10-16) defined B4/B8/B16/B32 *integer* widths; that note is superseded by this bf16 model. The cell PRD was updated to match on 2026-10-17 — the B4–B32 integer model is not planned (a companion integer mode is deferred, §7). One source per fact: this PRD owns the ALU's element model; the cell PRD links here rather than restating it.




*ALU-INFER datapath*

## 2.3 3. Function set (v1)

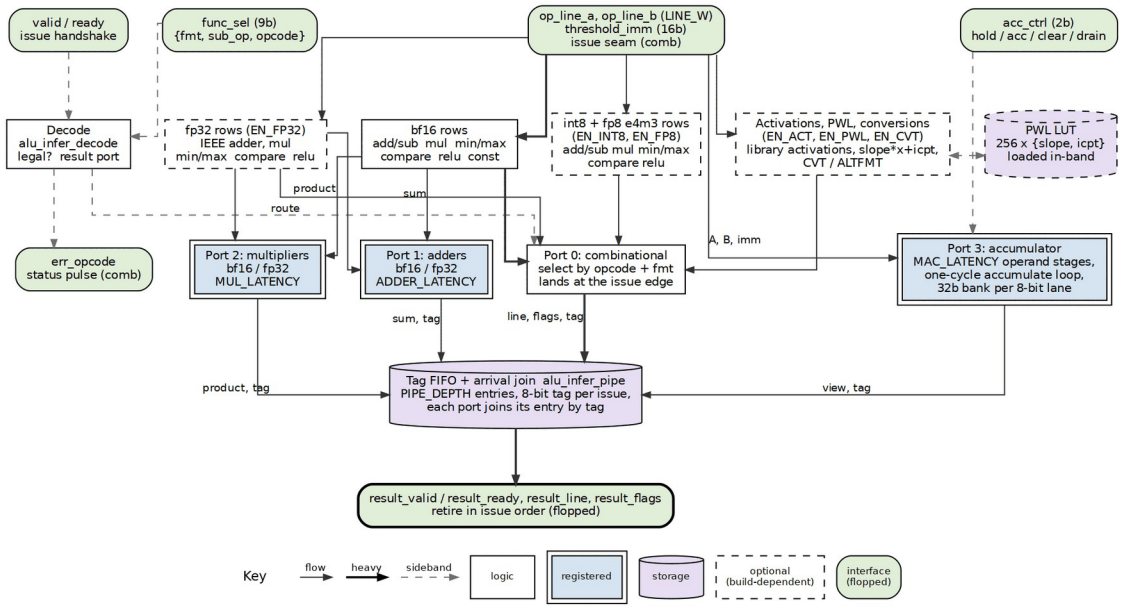
Opcode rows on the bf16 base; the `math_bf16_*` block named in each row is the implementation base in the ALU's RTL. Each row is an RTL table entry, not a new block.

| Opcode | Function | `rtl/math` base | Operand pattern |
|---|---|---|---|
| 0 | copy / passthrough | wires | 1 line → 1 line |
| 1 | add | `math_bf16_adder` | 2 lines → 1 line |
| 2 | sub | `math_bf16_adder` (sign-flipped operand) | 2 lines → 1 line |
| 3 | mul | `math_bf16_multiplier` (`math_bf16_mantissa_mult` + `math_bf16_exponent_adder`) | 2 lines → 1 line |
| 4 | mac | `math_bf16_fma` — fused, fp32 accumulate (§3 note) | 2 lines + accumulator → 1 line |
| 5 | min / max (sub-op 1b) | `math_bf16_min` / `math_bf16_max` (via `math_bf16_comparator`) | 2 lines → 1 line |
| 6 | compare / threshold (sub-op) | `math_bf16_comparator` | 2 lines or line + imm → 1 mask line |
| 7 | relu | `math_bf16_relu` | 1 line → 1 line |

**MAC accumulation (decided 2026-10-17):** the FMA row computes the bf16 product **exactly in fp32** (8×8 significand bits fit fp32's 24) and accumulates in a **per-lane fp32 accumulator**; one rounding to bf16 per result line at drain. This is the production bf16 pattern; it promotes the v0.2 open question to a decision.

Opcode space reserves 5 bits; sub-ops 2 bits; 8–31 are growth. The fused family (`math_bf16_fma`) makes the mac row the workhorse an inference workload expects.

The full block diagram — every v1 row with its `rtl/math` block, the fp32 accumulator, and the sideband plane:




*ALU-INFER block diagram*

**Growth rows** (same table, later opcodes — the library blocks already exist): leaky-relu / gelu / sigmoid / silu / tanh activations (`math_bf16_leaky_relu`, `math_bf16_gelu`, `math_bf16_sigmoid`, `math_bf16_silu`, `math_bf16_tanh`); reciprocal and divide (`math_bf16_reciprocal`, `math_bf16_newton_raphson_recip`, `math_bf16_goldschmidt_div`); exp2 / log2 scaling (`math_bf16_exp2`, `math_bf16_log2`, `math_bf16_log2_scale`); int8 drain (`math_bf16_scale_to_int8`); format conversions (`math_bf16_to_fp32`, `math_bf16_to_fp16`, `math_bf16_to_fp8_e4m3/e5m2`).

`FUNC_SEL` (cell PRD §7) carries {opcode 5b, sub-op 2b, element-width 2b} — the element-width field is **reserved** under this model (bf16 is the only v1 element width); its final disposition is §8.

### 2.3.1 Function-set roadmap (owner ruling 2026-10-19)

The v1 rows 0–7 in the table above stand unchanged. Row additions after FPGA bring-up, in priority order (all three implemented 2026-10-20 as opcodes 8, 9–10 and 11; MAS Chapter 2):

1. **reduce** — sum/max tree over a span of lines into the fp32 accumulator; unlocks softmax/layernorm/pooling.
2. **PWL/LUT activation** — 256-entry LUT + linear interpolation covering sigmoid/tanh/gelu/exp; exp plus reciprocal-multiply covers softmax's divide. No true divider is planned.
3. **const/bias** — emit the immediate register as a result line so programs can build bias lines without memory traffic.

Format roadmap, via the reserved `func_sel[8:7]` element-width field (open item M6): **int8** (4 lanes per 32-bit line) first — the same MAC fabric runs 4 MACs/clk and teaches quantization; **fp32** (1 lane per line) second — a general vector engine. **bf16 stays the v1 format.** Both implemented 2026-10-20 (`func_sel[8:7]` = 1 int8, 2 fp32), with fp8 e4m3 (§7) as format 3.

**No-fork rationale:** there are no A/B/C ALU variants — one ALU per cell; heterogeneity belongs at cell/tile granularity because delta/tile is payload-agnostic. The `alu-rv32i` and `alu-rv32f` blocks are ISA-datapath ALUs for the falcon cores (a different role, not variants of this one).

## 2.4 4. Interface contract

The contract the cell implementation obeys at the issue seam (the ALU↔️gatling seam; the MAS chapter “The Issue Seam” is the signal-level form, including the `err_opcode` status pulse added 2026-10-20):

| Signal group | Direction | Width | Notes |
|---|---|---|---|
| `op_line_a`, `op_line_b` | in | LINE_W each | operands — the function’s `OP_LINES` count is across both ports (1 for copy/relu: the A line only; 2 for binary/mac: the A line and the B line); both lines are presented **in the same clock** and captured at the issue edge (owner ruling 2026-10-17: two ops scheduled in the same clock finish at the same time) |
| `threshold_imm` | in | 16 (bf16) | immediate operand for opcode 6 |
| `func_sel` | in | 9b | {opcode, sub-op, reserved} — the legacy element-width field is reserved (§3) |
| `acc_ctrl` | in | — | accumulator clear / hold / accumulate (opcode 4 state) |

| Signal group | Direction | Width | Notes |
|---|---|---|---|
| `result_line` | out | LINE_W | one line per invocation for all v1 rows (2 bf16 lanes, fp32-rounded at drain for opcode 4) |
| `result_flags` | out | LINE_W | opcode-6 mask — one bit per lane, packed low (M5 closed 2026-10-17, owner ruling); zero for other rows |
| `valid` / `ready` | in handshake | 1/1 | elastic issue: the invocation enters the row pipeline when `valid && ready`; `ready` deasserts only under downstream backpressure, so a new invocation may be accepted **every clock** while the output drains (II=1); the issuer holds `valid` and its operands/config stable while `ready` is low |
| `result_valid` / `result_ready` | out handshake | 1/1 | elastic retire: each result is offered at its row's natural latency (MAS |

| Signal group | Direction | Width | Notes |
|---|---|---|---|
| | | | Table 2.6; the retired mailbox's uniform `ROW_LATENCY` is gone) and held with stable data until accepted; results retire strictly in issue order |

Invariants:

1. **Lane independence.** Except for opcode 4's accumulator, no lane of a result line depends on any other lane of the same or the other operand line. A lane can be checked against a scalar golden reference in isolation.
2. **Accumulator isolation.** The fp32 accumulator is per-lane, private to opcode 4, cleared only by `acc_ctrl`.
3. **Registered shell, uniform presentation, elastic timing.** Rows may be combinational or pipelined (cell MAS decides); the contract is a valid/ready elastic pipeline either way: every result is **presented** exactly `ROW_LATENCY` cycles after its issue, in issue order, when the output drains continuously. Quick rows compute in parallel at their natural speed and the block's alignment mechanism delays their presentation — no padded pipe stages on fast rows. **Tag-based alignment (owner ruling 2026-10-18):** when an invocation is loaded it takes an 8-bit tag, common to all ops that must execute in parallel; a FIFO tracks the tags in flight, and an op asserts its valid only when every op of its tag group is ready — the join yields group-aligned, in-order, elastic retirement without per-op timers. The v1-as-built block realizes the same contract with the alignment-timer mailbox of the MAS; the tag mechanism is the alignment architecture when rows gain uneven pipeline depths (§7). Backpressure propagates: a stalled consumer holds `result_valid` (data stable), freezes the alignment tracking, and deasserts `ready` at the input once the pipe fills; on release the pipeline resumes where it stopped. **Scheduler-paced feedback.** Whenever results feed back — opcode 4's chain today, any future recurrence — the scheduler owns the pace entirely: it shapes the `valid` stream and obeys `ready`, and

the ALU makes no autonomous pacing decisions (owner ruling 2026-10-17). **No tags on this seam** — the tags above live inside the block; seam flow control is the two valid/ready pairs alone, with no error signal. Steady state, un-stalled, is one invocation per clock regardless of depth.

4. **Function-table growth.** New opcodes and sub-ops are additive rows; they must not change §4's handshake or the lane-independence invariant.

## 2.5 5. Control interface — gatling issues the ALU; in-band everything

**Owner ruling 2026-10-19:** gatling sequences the four ALUs of its adjacent cells. The `gatling-rv32itetrad` barrel RV32I core runs one context per cell; a context's program presents operands to the cell's alu-infer issue handshake and retires result lines (Xcell POP/PUSH, gatling PRD §3). The program is the schedule. The ALU keeps its elastic valid/ready seam and tag-alignment contract (§4) unchanged, and makes no pacing decisions of its own.

The tile blocks take **all data, config, and programming in-band on delta** (owner ruling 2026-10-17). Configuration rides with the data as `cfg` packets (cell PRD §3) and the cell's demux writes its register map from the stream — **the stream is the programmer**. Consequences:

- **Gatling never originates config traffic.** There is no per-tetrad config broadcaster to design or verify.
- **One gatling per tetrad** (4 cells, four per tile): the tetrad is the tile's structural group, with one group-done per four cells, and four independent control points allow layer pipelining across tetrads without sixteen instruction streams. SIMD width per tetrad: 4 cells × 2 bf16 lanes = 8 bf16 lanes stepping together (16 int8 or fp8 lanes in those formats).
- **Xcell stays minimal** (custom-0): CFG/POP/PUSH/ECTX/FLUSH/KICK/WAIT, encodings in the gatling PRD v0.2. How POP/PUSH map onto this block's issue fields is gatling-MAS scope.
- **Observability stays with Ocelus.** The tile's MonBus aggregation and the SERV-based monitor plane are untouched; gatling adds per-context status to the observed set (gatling PRD §5). The ALU exposes no taps.
- **Not SERV, not a per-cell FSM:** SERV is the Ocelus agent core; the per-cell execution FSM of the first cell PRD is replaced by gatling (gatling PRD §5).

## 2.6 6. Verification hooks

- Stack per house convention: cocotb + rds-dv BFMs (cell PRD §9); the ALU's unit tests live in the cell's `dv/` tree since the RTL does.
- **Golden reference:** a small bf16 numerics model (fp32 computation per lane, rounded to bf16 RNE) — matching how the math library's own dv checks the bf16 blocks; lanes compared independently, which §4 invariant 1 makes a strong check. Opcode 4 checks the fp32-accumulate chain (long MAC runs, clear/hold corners, single-round-at-drain).
- **Special values are mandatory corners**, not extras: NaN payloads, ±inf, ±0, and denormals/FTZ behavior (the library's `SUBNORMAL_SUPPORT` parameter lineage — see the math references index) on every arithmetic row.
- **Activation rows** check against the library's reference approximations (piecewise/LUT), with the accuracy class stated per row in the cell MAS.
- **Cross-check against `rtl/math`:** the ALU's rows instantiate the library blocks; a unit-vs-library diff on identical operands is a bug in exactly one of them.
- **Issuer bring-up** is gatling's (its PRD §6: spike lockstep on RV32I, directed Xcell tests); the issue seam itself is verified here, with GAXI BFMs standing in for gatling.
- Coverage matrix: opcode (8) × special-value class × lane position (2 per line), plus accumulator chains and threshold immediate corners. The cell PRD's credit-loop tests already verify lines-in → credits-out independently of the datapath.

## 2.7 7. Deferred and growth

Status 2026-10-20: implemented — pipelining knobs (`MUL_LATENCY`, and `MAC_LATENCY` operand stages ahead of a one-cycle fused accumulate), activations, reciprocal, exp2/log2, int8 drain, conversions, fp8 e4m3 lanes. Not implemented, by this PRD's own rulings: divide (no true divider is planned, §3 roadmap), the B4–B32 companion mode, the CGRA replacement, and the cell-PRD items.

- **Deep-pipelined multiplier / FMA rows** — the combinational fp32 FMA (with its accumulator feedback) is the timing risk at FPGA bring-up; pipeline it with the adder's `PIPE_STAGE_*` pattern. Mixed row depths then stay II=1 with in-order retirement under the tag-based alignment ruling (§4 invariant 3), which "naturally takes care of the elastic nature" (owner 2026-10-18). A single lane's serial MAC dependency is arithmetic, not implementation — pipelining does not change it.
- **Activation expansion** (gelu, sigmoid, silu, tanh, leaky-relu) — library blocks exist; table rows once an owner priority calls for them.
- **Reciprocal / divide, exp2 / log2, int8 drain, format conversions** — library blocks exist (§3 growth list); same path.
- **fp8 element lanes** — `math_fp8_e4m3_*` / `math_fp8_e5m2_*` families exist; a mixed bf16/fp8 mode is an element-model extension, not v1.
- **Companion integer mode** (B4–B32) — deferred, and not currently planned; the bf16 model is the accelerator's single datapath (§2).
- **CGRA replacement** — per cell PRD §11, an array replaces this ALU behind the same issue handshake.
- Context switching / multi-function pipelines and cell-initiated grant requests (both cell PRD §11) — cross-referenced, not restated.

## 2.8 8. Open questions

- **NaN / inf semantics in min/max and activations.** IEEE 754-2008 `minNum/maxNum` quiet-NaN propagation vs the inference-convention "ignore NaN" — the library's `math_bf16_min/max` and `math_bf16_comparator` semantics decide; state the choice in the cell MAS.
- **Rounding and exception policy.** RNE throughout per the library; a dynamic rounding mode is out of scope (garuda's RNE ruling is the precedent). Whether the ALU reports bf16 flag equivalents (the library's blocks carry IEEE exception outputs) into `result_flags`, or leaves them unobserved, is a cell-observability decision.
- **Denormal / FTZ behavior** on the arithmetic rows — follow each library block's `SUBNORMAL_SUPPORT` default, or force one policy cell-wide?
- **Tetrad-controller boundary details** — the core's program-load path and local memory map, the kick/wait encoding, and whether it ever takes a MonBus observation tap (default: no — Ocelus/SERV owns observability). All land in the cell/tile MAS, not this PRD.
- **Immediate encoding** for opcode 6 (`threshold_imm` as a raw bf16 bit pattern vs a small int compared after convert).
- **Result-flag line format** for opcode 6 — **closed 2026-10-17** (owner ruling): per-lane 1-bit mask, packed low in the `LINE_W` line (M5).
- **`FUNC_SEL` element-width field disposition** — **closed 2026-10-20 by implementation:** `func_sel[8:7]` is the element-format field (0 bf16, 1 int8, 2 fp32, 3 fp8 e4m3), as the format roadmap planned (M6).

## 2.9 9. Dependencies

- `rtl/math` `math_bf16_*` family — the implementation base (§3 names the block per row): adder, multiplier (`mantissa_mult`, `exponent_adder`), `fma`, `comparator`, `min`, `max`, `relu`, plus the activation, divide, scale, and conversion blocks of the growth list.
- `cell/docs/PRD.md` — the cell around the ALU (memories, issue seam, packetizer, credits; reconciled to this PRD 2026-10-17).
- `gatling-rv32itetrad/` — the tetrad barrel core that issues the ALU (§5).
- `tile/docs/TILE_SPEC.md` §11 — packet, credit/grant, and coordinate protocols the cell (not the ALU) answers to; listed so a reader of this PRD can find the whole contract.
- `rtl/math/references/README.md` — the central math-paper index (bf16 format, activation, quantization, and fp8 citations for §§3, 7).

## 2.10 10. References

- `cell/docs/PRD.md` — the cell around the ALU (normative for everything except the ALU block, which this PRD owns).
- `gatling-rv32itetrad` PRD — the issuer: barrel RV32I + Xcell, one context per cell.
- S. Wang, P. Kanwar, "BFloat16: The Secret to High Performance on Cloud TPUs," Google Cloud blog, Aug. 2019 — the bf16 format's canonical definition (non-peer; citation-only in the math index). `rtl/math/references/README.md`.
- V. Nair, G. E. Hinton, "Rectified Linear Units Improve Restricted Boltzmann Machines," *Proc. 27th ICML*, 2010 — backs the opcode-7 ReLU row. PDF: `rtl/math/references/ReLU__Nair2010.pdf`.
- D. Hendrycks, K. Gimpel, "Gaussian Error Linear Units (GELUs)," arXiv, 2016 — backs the §7 gelu growth row. PDF: `rtl/math/references/GELU__Hendrycks2016.pdf`.
- S. Elfwing, E. Uchibe, K. Doya, "Sigmoid-Weighted Linear Units…" (*Neural Networks*, 2018) and P. Ramachandran, B. Zoph, Q. V. Le, "Searching for Activation Functions" (Swish, arXiv 2017) — back the §7 silu growth row. PDFs: `rtl/math/references/SiLU__Elfwing2018.pdf`, `rtl/math/references/Swish__Ramachandran2017.pdf`.
- B. Jacob et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," *Proc. IEEE CVPR*, 2018 — backs the §7 int8-drain row (`math_bf16_scale_to_int8`) and the §3 int8-lane roadmap. PDF: `rtl/math/references/Int8Quantization__Jacob2018.pdf`.
- D. Kalamkar et al., "A Study of BFLOAT16 for Deep Learning," arXiv, 2019 — backs the bf16 format choice (secondhand citation; primary provenance is the Google Cloud blog and the IEEE 754-2008 fp32 exponent). PDF: `rtl/math/references/README.md`.
- The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA — the custom-0 opcode space is the defined home for the gatling Xcell instruction subset (KICK/WAIT/…) described in §5. https://riscv.org/technical/specifications/

---

**Last Updated:** 2026-10-19 **Maintained By:** RTL Design Sherpa Project

# 3 GATLING PRD — Tetrad Compute Engine

**Version:** 0.2 (2026-10-19 — owner ruling: gatling sequences the four ALUs of its adjacent cells; the barrel core is the programmable sequencer for the tetrad; the cell's bf16 ALU remains the datapath.) **Date:** 2026-10-19 **Owner decision inputs:** "Instructions" (over MMIO) for fabric handshake; mini RV per tetrad (over per-cell ALU+FSM); codename gatling (barrel machine). **Supersedes:** PRD 0.1 wording that described gatling as replacing the per-cell ALU — the ALU stays; gatling replaces the execution FSM as its driver.

---

## 3.1 1. Purpose

Give each tetrad of the compute tile a programmable engine small enough to instantiate four times per tile, powerful enough to run compiled programs over cell input data, and honest enough to teach: the barrel processor as the kestrel→merlin lesson, inverted — instead of pipeline stages hiding latency for one stream, N contexts hide the multi-cycle latency of one shared datapath.

Owner ruling 2026-10-19: gatling sequences the four ALUs of its adjacent cells. The barrel RV32I core is the programmable sequencer for its tetrad — each context runs a small program whose POP presents operands to the cell's alu-infer issue handshake and whose PUSH retires a result line; the ALU keeps its elastic valid/ready seam and tag-alignment contract unchanged. *The program is the config*. Mode bits that remain (element width, credit batch N) move to the CFG read path. Gatling is also a barrel RISC-V with custom commands (Xcell: CFG/POP/PUSH/ECTX/FLUSH/KICK/WAIT) — the program is the schedule. Gatling never originates config traffic; cell configuration arrives in-band over delta.

## 3.2 2. Non-goals

- Not a falcon-suite rung. No performance chase; IPC-per-context ~ 1 is the target, not a benchmark result.
- Not per-cell processors. One engine per tetrad; the four cells are the four contexts.
- No caches, no virtual memory, no interrupts/CLINT, no M/A/F. (Machine state is 4× {PC, x1..x31} + mode CSR shadow. Full stop.)
- No CSRs as a subsystem — `Zicsr` deliberately absent; config reaches the program through the Xcell `CFG` instruction.

## 3.3 3. ISA — RV32I + Xcell

Base RV32I exactly as kestrel implements it (same opcode map, same trap/halt contract) so the spike lockstep flow and rv32ui battery carry unchanged. The Xcell extension occupies the custom-0 opcode (`0001011`, 0x0B) — reserved space, zero collision risk with future standard extensions.

### 3.3.1 3.1 Xcell encoding (custom-0, opcode 0x0B)

```
 31                      15 14   12 11      7 6       0
+-------------------------+-------+---------+--------+
|        imm[31:12]       | funct3|   rd    | 0x0B   |  CFG
|        000000000000     |  000  |   rd    | 0x0B   |  POP
|        000000000000     |  001  |  rs1    | 0x0B   |  PUSH
|        000000000000     |  010  |  00000  | 0x0B   |  ECTX
|        000000000000     |  011  |  rs1    | 0x0B   |  FLUSH
|        000000000000     |  100  |  rs1    | 0x0B   |  KICK
|        000000000000     |  101  |  rs1    | 0x0B   |  WAIT
+-------------------------+-------+---------+--------+
funct3 110/111 : reserved.
funct3 010 (ECTX) and 011/100/101 (FLUSH/KICK/WAIT) added in v0.2.
The v0.1 WAITCD/WAITGT placeholders are superseded by KICK/WAIT
pending
the tetrad MAS.
```

| Instr | Action | Blocking behavior |
|---|---|---|
| `CFG rd, imm` | rd ← mode-register[imm]: 0=element mode (0=bf16: 2 bf16 lanes per 32-bit line, fp32 accumulate; other values reserved for the int8/fp32 roadmap per alu-infer PRD `func_sel[8:7]`), 1=credit batch N, 2=context ID (read-only, see §7 open item), 3+=reserved | never blocks |
| `POP rd` | rd ← next element of this context's input memory; head advances; consumed-line credit returned to rapids per credit-batch config | context stalls if input empty — hardware interlock, not a spin |

| Instr | Action | Blocking behavior |
|---|---|---|
| `PUSH rs1` | rs1 → this context's result memory; tail advances; grant requested on result path; packetizer batches X flits | context stalls if no grant |
| `ECTX` | context done: freeze PC, raise done status to tetrad MonBus, release datapath share; context restarts on reset/config reload | drains at next context boundary |
| `FLUSH rs1` | rs1[14:0] → cell `FLUSH_DEST` {ext,level,X,Y,Z}; rs1[18:15] → cell `FLUSH_TAG` (RAPIDS sink-channel tag); trigger cell flush-and-send of result memory | context stalls until drain controller accepts command |
| `KICK rs1` | raise tetrad coordination event on the tetrad MonBus with value `rs1`; provisional encoding pending tetrad MAS | non-blocking once event is posted |
| `WAIT rs1` | stall context until the tetrad coordination condition in `rs1` is observed (e.g. group-done bitmask); provisional encoding pending tetrad MAS | context stalls until condition true |

Design note (owner doctrine): fixed hardware owns the line-rate handshake (credit/grant detection, memory ports); the program owns policy. POP/PUSH blocking interlocks subsume most waiting — a context that would wait simply isn't scheduled. WAIT is an explicit escape hatch for cross-context coordination, not the common case.

Element semantics: with element width < 32, POP zero-extends into rd and PUSH takes the low bits of rs1; packing/unpacking is the program's business (one ANDI/SLTI pair, honest and visible).

### 3.3.2 3.2 Assembly mnemonics

`gatling_pop`, `gatling_push`, `gatling_cfg`, `gatling_ectx`, `gatling_flush`, `gatling_kick`, `gatling_wait` as canonical toolchain names; assembler aliases `pop/push/cfg/ectx/flush/kick/wait` live in the Xcell binutils fragment when the flow arrives (§8).

## 3.4 4. Barrel microarchitecture

### 3.4.1 4.1 The model

Kestrel's single cycle is PC + regfile-read + ALU + memory in one combinational path — the teaching point and the timing/area cost. Gatling splits that into a 4-state FSM:

```
F  : address = ctx[sel].pc; fetch word from ctx program image
D  : decode; read ctx[sel] regfile (rs1/rs2 by decoded index)
X  : ALU / Xcell unit (POP/PUSH/CFG execute here; interlock check)
W  : writeback to ctx[sel] regfile; pc advance (or branch target)
```

Each FSM state advances context `sel` one step; `sel` rotates 0→1→2→3→0. Steady state: every context retires one instruction per 4-cycle window ⇒ 1 IPC per context, one gatling ALU, one decoder, one Xcell unit, **four** register files + PCs (the cheap replication) — replacing the cell PRD's four execution FSMs while each cell keeps its own bf16 ALU datapath (owner ruling 2026-10-19).

### 3.4.2 4.2 Structural hazards (all resolved by rotation, by construction)

- ALU/decoder/memory-port conflict: impossible — one user per state.
- Input/result memory port conflict across contexts: impossible — each context owns its cell's memories, addressed by `sel`.
- Interlock stall (POP empty / PUSH no-grant): only `sel` stalls; rotation skips it (bubble) — three contexts keep the datapath busy.

### 3.4.3 4.3 Branch behavior

Predict-not-taken, resolved in X (kestrel semantics carry: the branch decision is one ALU comparison). A taken branch costs the rotating window one bubble for that context — no flush machinery, no BTB. The predictor study belongs to peregrine, not here.

### 3.4.4 4.4 Halt/trap

No traps (no exceptions in base scope; illegal instruction → context-local lock with status bit via MonBus, other contexts continue — a tetrad is not a CPU, one bad cell must not sink the tile). `ECTX` is the normal exit. Reset is per-tile from the tile root; program load via the fabric config path (PKT_CONFIG) into each cell's program image region — kestrel's `mem_loader`/AXIL pattern becomes the in-band loader, per the standing in-band-only fabric decision.

## 3.5 5. Contract fit (cell PRD 0.1, unchanged interfaces)

| Cell PRD block | Stand / change |
|---|---|
| delta_mesh_cell node, containment | stands — gatling sits behind the node's local port |
| Input memory, line fill by node from rapids | stands; per-context, addressed by `sel` |
| Result memory, X-flit batching, packetizer, grants | stands |
| Consumed-line credits to rapids, batch N (default 1) | stands; N read via CFG |
| Element widths B4/8/16/32 | superseded — alu-infer's bf16 element model (2 bf16 lanes per 32-bit line, fp32 accumulate) is normative; CFG[0] reports the alu-infer element mode (bf16 today; int8/fp32 modes are a roadmap item in the alu-infer PRD via `func_sel[8:7]`) |
| ALU + execution FSM (config-selected fn) | **replaced by gatling as the execution FSM driver** — the cell's bf16 ALU datapath remains; gatling issues it via POP/PUSH and the alu-infer elastic valid/ready seam (owner ruling 2026-10-19) |
| MonBus observation (per-cell → per-tetrad merge) | stands; gatling adds per-context status (active/stall-reason/done) to the observed set |

Consequences that must not fork: credit counting is in *elements consumed* (POP), not instructions; grants are in *elements pushed* (PUSH), not flits — flit batching stays inside the packetizer.

## 3.6 6. Verification

- **Base ISA:** spike lockstep on RV32I exactly as kestrel does it — reuse the rds-dv flow; rv32ui battery is the gate (42/42 precedent).
- **Xcell unit:** directed cocotb — POP empty-stall / credit batch N=1,2,4; PUSH grant/backpressure; CFG read of mode shadow; ECTX done/observability; element-width zero-extend and slice.
- **Barrel properties:** cocotb assertions — one retirement per context per rotation window; stall of context k never stalls j≠k; rotation fairness (round-robin, no starvation with all-runnable).
- **Tetrad integration:** the eventual `tetrad.sv` TB drives four input-memory streams, checks per-context result memories and the credit/grant ledger against the cell PRD reference model.
- **Formal (where it pays):** deadlock freedom of the interlock (POP-empty ∨ PUSH-no-grant ∧ no ECTX ⇒ eventually credit ∨ grant), following the repo's targeted-formal doctrine.

## 3.7 7. Open items (owner decisions needed)

1. **ALU-infer relationship:** RESOLVED 2026-10-19 — gatling is the programmable sequencer for the tetrad's four bf16 ALUs; each context runs a small program whose POP presents operands to the cell's alu-infer issue handshake and whose PUSH retires a result line; the ALU keeps its elastic valid/ready seam and tag-alignment contract unchanged. The v0.1 wording that made gatling "the ALU
   - execution FSM" is superseded.
2. **Context visibility:** implicit (hardware-only `sel`; software cannot tell contexts apart — simplest, binaries uniform) vs explicit (CFG[2] returns context ID, enabling per-context specialization). Leaning implicit; one word settles it.
3. **Program image placement:** input memory holds both program and data (self-contained cell, POP-after-code addressing) vs separate program region (needs a second read port or F-state fetch from program image, D-state POP from data). Separate is cleaner; self-contained is cheaper. Open.
4. **Illegal-instruction policy:** context-lock (proposed §4.4) vs tetrad-lock. Leaning context-lock.
5. **Xcell binutils/toolchain:** hand-assembled hex for bring-up; gcc/binutils fragment later. Non-blocking.

Resolved/added in v0.2: the FLUSH/KICK/WAIT Xcell encodings of §3.1 (answer TILE_SPEC §11.14 and the alu-infer PRD §5 tetrad-coordination pair). Their exact field layouts and MonBus integration remain provisional pending the tetrad MAS.

## 3.8 8. Deliverables & sequencing

1. `docs/barrel_model/` study chapter (kestrel cost → interleave fix) — writes first, it is the justification.
2. `gatling_has` + ISA appendix (Xcell table as authority).
3. RTL: fubs (kestrel-derived) → `gatling_top` barrel + Xcell unit → flists; unit TBs per fub on repo rails.
4. Spike lockstep bring-up of the RV32I subset; then Xcell directeds.
5. Tetrad integration when `tetrad.sv` lands in `tile/`.

## 3.9 9. History

- 2026-10-19: PRD 0.2. Owner ruling: gatling sequences the four ALUs of its adjacent cells — the barrel core is the programmable sequencer for the tetrad, the cell's bf16 ALU remains the datapath, and gatling never originates config traffic. Added FLUSH/KICK/WAIT Xcell encodings; fixed POP to read `rd`; updated CFG[0] to report the alu-infer element mode.
- 2026-10-17: PRD 0.1 scaffold. Owner decisions: instructions over MMIO; mini RV per tetrad; codename gatling; four contexts = four cells; blocking POP/PUSH interlock; ECTX exit. Cell PRD 0.1 contracts confirmed unchanged outside the replaced ALU/FSM block.

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

**← Back to cell**

# 4 Cell — Compute Engine IP Product Requirements Document

**Version:** 0.5 (2026-10-19): records the 2026-10-19 owner ruling that **gatling sequences the cell ALU** (POP/PUSH/FLUSH Xcell encodings, gatling PRD v0.2) while the slice-3 `cell_exec_fsm` remains the standalone bring-up driver; DEST is explicitly reserved for the deferred cell→cell path and FLUSH_DEST/FLUSH_TAG are the operational drain destination; strap terminology replaces remaining grant terms in §§6.3/7/10 (slice-1 RTL still uses the old grant identifiers, noted in the MAS). Carries v0.4 (2026-10-18): §6.3 rewritten to the strap model (TILE_LAND_FLITS, RAPIDS returns on drain; owner direction 2026-10-18) and the result destination becomes gatling-programmed (TILE_SPEC §11.14) — DEST chain-back superseded, register retained for the deferred cell-destined path. Carries v0.3 (2026-10-18): **cell register file** + **sender-generic credits** — owner direction 2026-10-18 defines the cell as the lowest module: the alu-infer ALU, one delta node, the config registers, the ALU registers (A/B/C/D + T) as persistent operands for repeated calculations over a run (OP_SRC operand-source mux, IP/OP memory pointers), and the data-in / data-out memories; the tetrad engine is deferred (the owner-authored gatling-rv32itetrad barrel-core PRD is the leading candidate; §10). Credit returns become sender-generic — per-source batching to the recorded source coordinate; the packetizer gains DEST (default: recorded source, chain-back). §4 reconciles to the alu-infer elastic valid/ready pipeline (“one-function-at-a-time” superseded). The house-style graphviz figure replaces the ASCII block diagram. Carries v0.2: ALU element model, function set, and control interface reconciled to the alu-infer PRD v0.3 (owner direction 2026-10-17; owner datapath decisions 2026-10-16). **Status:** Pre-HAS PRD. This document is the cell’s product definition; the HAS/MAS follow it. The tile architecture it plugs into is the adopted hierarchy of `tile/docs/TILE_SPEC.md` §11 (owner decision 2026-10-16); the packet header, payload types, coordinate, and credit/grant protocols referenced below are defined there and are not restated except where the cell consumes them.

---

## 4.1 1. Identity and scope

The **cell** is the tile's compute unit: sixteen of them make a tile. Each cell contains exactly **one delta node** — a `delta_mesh_cell` instantiated inside the cell, its local port facing the cell's internal datapath and its seven other ports passing through to the cell boundary. That containment is the **entire** cell↔delta relationship (owner clarification 2026-10-16): the node is a black-box subcomponent governed by delta's own HAS/MAS, and everything else in the cell — the memories, the execution FSM, the ALU, the ALU registers, the packetizer, the credit logic, the registers — is the cell's own design, speaking the tile-level contracts of TILE_SPEC §11, not delta internals. The node fills the cell's input memory from delta traffic; the cell's ALU and execution FSM consume that memory and produce results; results leave as batched flits back through the node toward their destination. As each input line is consumed, a credit returns to the line's sender so it may write more.

Owner decision 2026-10-16: the cell's compute engine is the **ALU + execution FSM** described here. The earlier CGRA-array ambition (README, 2026-10-13) is **deferred** — §11 — and the `axon_spec.md` sketch (SERV-based result collector) is **superseded**: the result collection, packetization, and streaming it proposed are this PRD's output path, and its interfaces section predates the adopted §11 header/credit protocols.

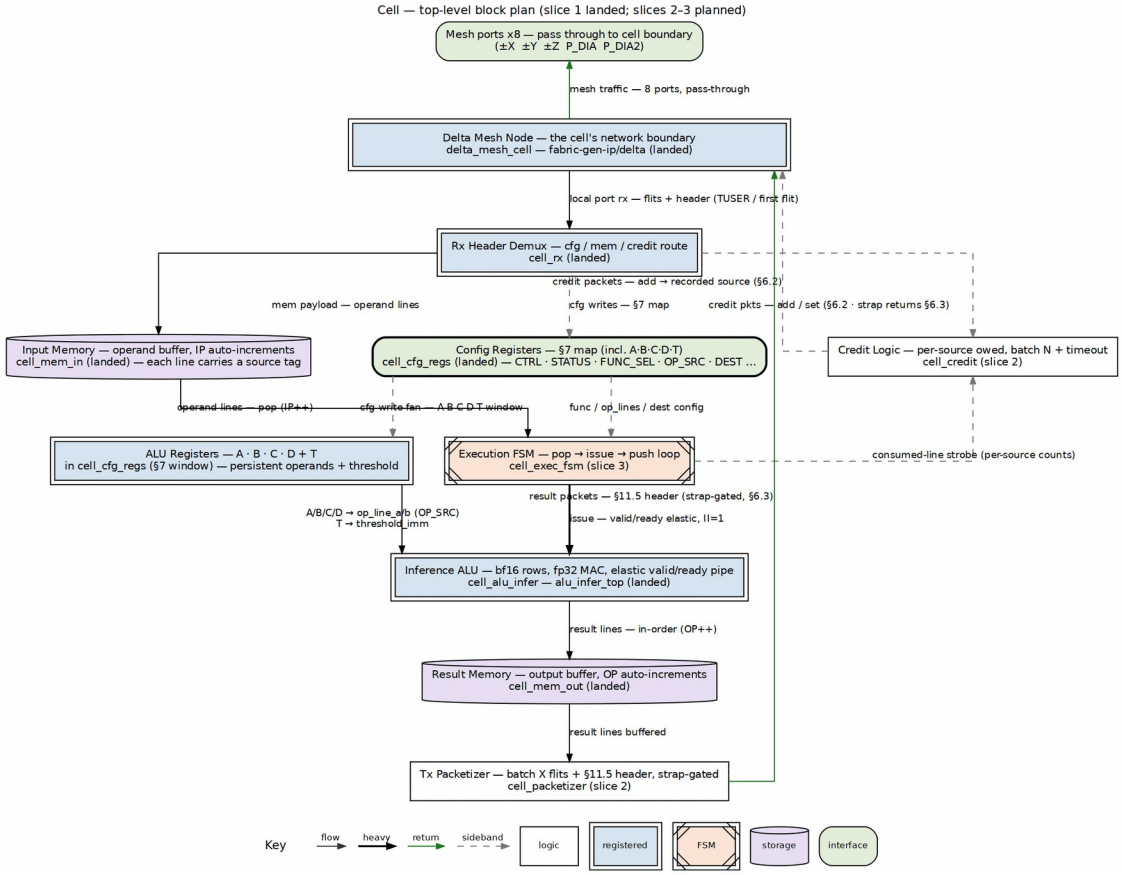
Owner direction 2026-10-17: the ALU block is governed by the `alu-infer` PRD — bf16 element model on the `rtl/math` `math_bf16_*` library, fp32 MAC accumulation, and a per-tetrad RV32I control core (not SERV; SERV is the Ocelus observability agent). Config and programming arrive in-band with the delta traffic; the tetrad core is a local reaction agent, not a config originator. See alu-infer PRD §5.

Owner direction 2026-10-18: **the cell is the lowest module and is defined first.** Its contents: the alu-infer ALU, one delta node, the config registers (§7), the ALU registers A/B/C/D (+T) — persistent operands for a sequence of the same calculations over a run — the data-in memory, and the data-out memory. The tetrad-level engine is **deferred**: gatling/tetrad is figured out once the cell is fully defined and working, with the owner-authored `gatling-rv32itetrad` barrel-core PRD (2026-10-17) the leading candidate. See §10.

Out of scope: tetrad/root node internals, rapids internals, the CXS modules themselves, and fabric-level Z stacking.

## 4.2 2. Architecture

### 4.2.1 Figure 1: Cell top-level block plan (house style — three flows: programming down the center, operand fill on the left, results and credits back up the right)




*Cell top-level block plan*

| Block | Function | Notes |
|---|---|---|
| `delta_mesh_cell` | network attachment | instantiated black box, not owned, here — containment is the whole cell↔️delta relationship (owner clarification 2026-10-16); the cell consumes its local port and passes its mesh ports through; delta HAS/MAS |

| Block | Function | Notes |
|---|---|---|
| | | govern only the node |
| rx header/demux | parses the §11.5 tile header (first data flit) and delta's route header (TUSER on AXIS), routes payload to input memory, handles `cfg` writes to registers, routes `credit` packets to the credit logic | per §11.6 (TILE_SPEC v1.6 clarification: tile header = first data beat on both transports; cell-bound traffic rides delta QoS ≠ 0 — QoS 0 is the node's own config plane); records each packet's tile-header source coordinate |
| input memory | delta-written operand/input buffer | sized in lines; its flit capacity is the boot advertisement value; each line carries a source-coordinate tag for credit returns (§6.2); read pointer `IP` auto-increments on pop |
| config registers | AXIL, registers-only per house rule | §7 — `CTRL`/`STATUS`/`FUNC_SEL`/`OP_SRC`/`DEST`/batching knobs/`IP`/`OP` |
| ALU registers | `A`·`B`·`C`·`D` persistent operands + `T` threshold | §7; written by `cfg` packets or AXIL, read into the ALU operand muxes per `OP_SRC`; persistent across a run (owner direction 2026-10-18) |
| credit logic | returns credits to the recorded sender of each consumed input line | §6.2; per-source batching, N configurable, default 1 |
| ALU + execution FSM | pops line(s), issues them to the ALU's elastic | §4; the cell's only datapath |

| Block | Function | Notes |
|---|---|---|
| | pipeline, pushes result line(s) | |
| result memory | buffers result lines before packetization | sized with §12; write pointer `OP` auto-increments on push |
| packetizer | batches result writes into X flits, builds §11.5 header (destination per §6.3 — gatling-programmed drain), sends via node | sends only against the `TILE_LAND_FLITS` strap counter (§6.3) |

## 4.3 3. Dataflow

A run is programmed first, then fed; everything — data, config, and programming — arrives in-band (owner ruling 2026-10-17):

1. **Program.** `cfg` packets (`ptype` = `cfg`) write the §7 registers — `FUNC_SEL`, `OP_SRC`, `DEST`, the persistent operands `A`–`D`, `T`, and the batching knobs — instead of the input memory. A `cfg` read is answered with a write-format packet echoing the tag, per §11.8.
2. **Fill.** A write packet (`ptype` = `mem`, destination coordinate `{level=0, cell X, Y, Z}`, target address inside the cell's input-memory window) arrives on the node's local port — from rapids in the v1 flow, but sender-generic by construction: any node may send it. The demux writes payload lines into the input memory and tags each line with its packet's source coordinate. The sender spends one end-to-end credit per flit sent, header included, from the per-destination counter it holds for this cell (§11.9).
3. **Consume.** The execution FSM pops the configured number of input lines for its function (`IP` auto-increments), issues the invocation to the ALU, and the elastic pipeline returns result line(s) in order; the FSM pushes them into the result memory (`OP` auto-increments).
4. **Credit return.** As each input line is consumed, the credit logic owes the line's recorded sender one credit (§6.2 — per-source batching), so the sender refills the space. Credits return on consumption, not arrival (§11.9(3)).
5. **Emit.** When the packetizer has ≥ X result flits (or a flush condition fires, §6.3), it builds the §11.5 header — destination = the gatling-programmed drain destination (§6.3; the cell does not choose it) — source = this cell's external ID, `op` = 0, length = bytes that follow, tag = the sink channel gatling programmed — and sends the packet through the node, spending its `TILE_LAND_FLITS` strap counter (§6.3). Cell-destined sending awaits the tile-spec extension (§6.3).

## 4.4 4. ALU and execution FSM

The FSM is a pop→issue→push loop: it issues invocations into the ALU's valid/ready **elastic pipeline** (multiple invocations in flight, results retire in order — the alu-infer MAS contract), and pushes each retired result line into the result memory. Functions are register-selected, and both the function and its operand pattern come from config, so new functions are RTL table rows, not new blocks. (The v0.2 "one-function-at-a-time" phrasing is superseded by the elastic contract; owner ruling 2026-10-17, alu-infer PRD v0.4+.)

**Operand sources (owner direction 2026-10-18):** the ALU's `op_line_a` / `op_line_b` are driven through an operand-source mux configured by `OP_SRC` (§7): each side selects the input-memory pop or one of the persistent ALU registers `A`–`D` (32 bits each, two bf16 lanes — matching `LINE_W`), and `T` (bf16) backs `threshold_imm`. This is the "sequence of the same calculations over a run" pattern: streamed operands arrive from delta into the input memory and pop under `IP`, while persistent scalars (weights, biases, constants) sit in `A`–`D` for the whole run. `IP`/`OP` auto-increment on pop/push and wrap at the configured memory depth; `CTRL.soft_reset` clears them.

**Element model (owner direction 2026-10-17, superseding the B4/B8/B16/B32 note of 2026-10-16):** a 32-bit line is read as **2×bf16** elements, and the function array is built mostly from the `math_bf16_*` family of `rtl/math` (adder, multiplier, FMA, comparator, min/max, ReLU, activations as growth). The ALU's product definition — element model, function set, interface contract, verification doctrine, and the per-tetrad control interface — is the **`alu-infer` PRD**, which governs the ALU block; it is not restated here (one source per fact). MAC accumulates in fp32 with one rounding per result line at drain. Wider lines (§12) scale the lane count, never the element width.

**Gatling sequences the cell ALU (owner ruling 2026-10-19):** in the tetrad-integrated tile, gatling (the barrel-core tetrad engine) drives the cell's alu-infer ALU through `Xcell` instructions: `POP` presents operand line(s) to the ALU issue handshake, `PUSH` retires result line(s) into the result memory, and `FLUSH` programs `FLUSH_DEST`/`FLUSH_TAG` and triggers the drain. The slice-3 `cell_exec_fsm` remains the standalone bring-up driver with the same POP/issue → PUSH/retire client contract. Gatling never originates cell config traffic; cell programming stays in-band per the 2026-10-17 owner ruling. The `Xcell` encodings (`CFG`, `POP`, `PUSH`, `ECTX`, `FLUSH`, `KICK`, `WAIT`) live in gatling PRD v0.2.

**v1 function set:** eight opcode rows (copy, add, sub, mul, mac, min/max, compare/threshold, relu) defined with their `math_bf16_*` implementation base in alu-infer PRD §3; not restated here. Opcode space reserves 5 bits; 8–31 are growth. Exact datapath widths and latency land in the MAS, consistent with the math area's learning ethos (sophisticated low-latency techniques).

## 4.5 5. Packet interface

The cell consumes and produces the §11.5 header unchanged; §11.6 locates it per transport. Fields the cell uses directly:

| Field | Cell use |
|---|---|
| Destination coordinate | demux match against `{level=0, own X/Y/Z}`; rapids-bound emits use `FLUSH_DEST` (gatling-programmed). `DEST` is reserved for the deferred cell→cell path (TILE_SPEC §10). |
| Source coordinate | the tile header's source field (the originator — TILE_SPEC v1.6 §11.5 note; not delta's hop-source route header), recorded per arriving packet — credit-return destination and the deferred chain-back default for `DEST` (rapids sends as `level` 3, §11.3) |
| `ptype` | `mem` → input memory; `cfg` → registers; `credit` → credit/grant handler |
| `op` | write vs `cfg` read; credit `add` vs `set` |
| Length | payload bytes / credit count / read size |
| Target address | input-memory line index or register offset |
| Tag | echoed on replies and result packets so rapids routes them to the right sink channel |

## 4.6 6. Credit behavior

### 4.6.1 6.1 Boot advertisement

After reset the cell sends one `set` credit packet (`op` = 1) to `level` 3 carrying its input memory's size **in flits**. Rapids's per-destination counter for this cell starts at 0 and cannot send until the advertisement arrives — the advertisement is the cell's "alive" (§11.10). The packet rides the credit VC and needs no grant.

### 4.6.2 6.2 Consumed-line credit returns

- Each input line consumed by the FSM owes **its sender** one credit return. The sender is the source coordinate recorded when the demux wrote the line (every input-memory line carries a source tag). Rapids (`level` 3) is the sender in the v1 flow, but the return is sender-generic per §11.9 — a credit return goes to the source coordinate of the packet whose flits it frees, so a cell that streamed into this cell's input memory receives the credits.
- Returns are `add` credit packets addressed to the recorded source coordinate, per §11.9(4): routed like any packet by destination coordinate; the sink port accepts them unconditionally and never stores them. They ride the credit VC and need no grant.
- **Batching (owner decision 2026-10-16, extended sender-generic 2026-10-18):** the credit logic keeps a per-source owed count and sends one credit packet per N consumed lines **per source**, with a timeout so a partial batch drains. **N = 1 is the default and the exact per-line behavior.** N > 1 trades return latency for less credit-packet traffic on the fabric.

### 4.6.3 6.3 Straps on the result path

The result path adopts §11.9's strap mechanism (owner direction 2026-10-18: credits toward RAPIDS are **straps, not grants**):

1. Toward RAPIDS the cell keeps no grant protocol. Space in RAPIDS's sink is fixed by construction: a per-cell strap of `TILE_LAND_FLITS` flits (a Hive-set parameter passed to RAPIDS, default 16 — TILE_SPEC §11.9 Hive contract), reserved at configuration.
2. The packetizer sends a result packet only while its strap counter covers the whole packet, header included, and spends the counter as it sends. RAPIDS returns `credit`/`add` as it drains the cell's landing space, restoring the counter.

3. Resync/re-baselining of the strap state follows RAPIDS's advertisement/resync rules (TILE_SPEC §11.10); exact timing is a tile §10 open item.
4. The cell never asks for result space — no ask path; the strap is the contract.

**Result destination (owner direction 2026-10-18, TILE_SPEC §11.14):** gatling hands the cell the drain destination. Result flushes go to `{ext 0, level 3}` (the tile I/O), source = this cell's external ID, tag = the sink channel gatling programmed. **The cell does not choose the destination.**

- **Operational drain:** `FLUSH_DEST` (coordinate) and `FLUSH_TAG` (sink channel) are programmed by gatling and used by the packetizer when `CTRL.flush-and-send` fires.
- **Reserved/deferred:** the legacy `DEST` register (offset `0x38`) is retained only for the future cell→cell result path. It is **not** used for rapids-bound drains. `DEST` becomes operational only after the tile-spec cell→cell credit/grant extension lands (TILE_SPEC §10); until then, v1 result traffic is RAPIDS-bound and `FLUSH_DEST`/`FLUSH_TAG` take precedence.

## 4.7 7. Config registers

AXI4-Lite, registers-only (house rule); routing/mode config travels in-band as `cfg` packets per the standing owner decisions — every register below is writable both from AXIL and from the `cfg` stream, and readable over AXIL for debug. v1 map:

| Offset | Register | Notes |
|---|---|---|
| 0x00 | `CTRL` | enable, soft reset (also clears `IP`/`OP`), flush-and-send (packetizer) |
| 0x04 | `STATUS` | busy, input-empty, result-nonempty, strap counter nonzero (slice-1 RTL alias `status_grant_nonzero`) |
| 0x08 | `FUNC_SEL` | opcode (5b) + sub-op (2b) + reserved (2b) — element width is reserved under the bf16 model (alu-infer PRD §3) |
| 0x0C | `OP_LINES` | input lines per function invocation |
| 0x10 | `EMIT_BATCH_X` | flits per result packet (packetizer) |
| 0x14 | `CREDIT_BATCH_N` | consumed lines per credit packet **per source**, default 1 |
| 0x18 | `IRQ_EN` | done, error, strap-starved (slice-1 RTL alias `IRQ_GRANT_STARVED_BIT`), credit-timeout enables |
| 0x1C | `IRQ_STATUS` | write-1-clear status |
| 0x20 | `A` | persistent operand register A — 32b (2×bf16 lanes); feeds `op_line_a/b` per `OP_SRC` |
| 0x24 | `B` | persistent operand |

| Offset | Register | Notes |
|---|---|---|
| | | register B |
| 0x28 | C | persistent operand register C |
| 0x2C | D | persistent operand register D |
| 0x30 | T | bf16 threshold immediate backing register (opcode 6) |
| 0x34 | OP_SRC | operand-source mux: sel_a[2:0], sel_b[2:0] — 0 = input memory (pop), 1 = A, 2 = B, 3 = C, 4 = D; reset 0 (both operands from memory) |
| 0x38 | DEST | *reserved for the deferred cell→cell path (2026-10-18/19 ruling)* — the rapids-bound drain destination is gatling-programmed into FLUSH_DEST/FLUSH_TAG; the cell does not choose it. DEST is retained (bits [13:0] = §11.3 coordinate at COORD_W = 4; bit [16] = override) but is **not** used until the tile-spec cell→cell credit/grant extension lands (TILE_SPEC §10). |
| 0x3C | IP | input-memory read pointer — auto-increments on pop, wraps at depth |
| 0x40 | OP | result-memory write |

| Offset | Register | Notes |
|---|---|---|
| | | pointer — auto-increments on push, wraps at depth |
| 0x44+ | counters | lines in, lines out, credits returned, packets sent (§8) |

The A–D/T window is written by `cfg` packets like any other register (§3); loading A–D *from* the input memory is deferred (§11).

## 4.8 8. Status and observability

- IRQs: computation done, error, strap-starved (a packetizer with data and no strap, time-limited), credit-timeout.
- Software counters per §7.
- One MonBus observer stream per cell, on AXI4-Lite (owner direction 2026-10-19), per the tile's §5 aggregation hierarchy: the cell's stream is a leaf the tetrad-level merge consumes, alongside the other three cells and the tetrad node. Exact leaf selection is the tile §10 open item.

## 4.9 9. Verification hooks

cocotb + rds-dv BFMs per the standing convention. Test levels follow the house gate/func/full grid. Natural hooks: drive the node from a GAXI master instead of a mesh (the node's local port is a normal stream face), force credits and grants from the TB, and check the credit loop (count lines in → credits out, per recorded source) independently of the datapath.

## 4.10 10. Dependencies

- delta — the backbone network — **in progress**, built by a separate agent (fabric-gen-ip/delta). The cell instantiates one `delta_mesh_cell` node as a black box; modes 9–10 diagonal ports landed (Task 6c), the rest of the backbone is being built out.
- rapids — the DMA running data in and out of the tile — **in progress**, separate lane. The cell's dependencies on it are the per-destination credit counters, strap pool (`TILE_LAND_FLITS`), and sink-to-source credit-return path (§11.9 requirements list); sequencing with the rapids lane.
- rapids_cxs + delta CXS migration — **in flight** (amba lane); the cell's §11.6 transport note applies unchanged until delta ports move.
- TILE_SPEC §10 open items that touch the cell: input-buffer sizing, grant allocation policy, resync timing, cell→cell grant mediation (needed for cell-destined `DEST`, §6.3).
- the alu-infer PRD — the ALU block's product definition (bf16 element model, function set, interface contract, control interface); **normative for the ALU** since 2026-10-17.
- the alu-infer MAS — the implementation-intent book (module tree, function rows, accumulator, interface, cycle behavior, verification, configuration); lives in this directory's `docs/alu_infer_mas/`.
- a tetrad-level RV32I control core per alu-infer PRD §5 — **deferred** (owner direction 2026-10-18): the tetrad engine is defined once the cell is fully defined and working, with the owner-authored `gatling-rv32itetrad` barrel-core PRD the leading candidate and kestrel-class a candidate flavor via `CORE_FLAVOR`. When it lands, gatling sequences the cell ALU through `POP`/`PUSH`/`FLUSH` (owner ruling 2026-10-19); the cells see its local AXIL register plane and its operational writes only. Cell *programming* stays in-band (owner ruling 2026-10-17).

## 4.11 11. Deferred

- CGRA array of richer PEs (2026-10-13 ambition): the cell's block boundaries — node, input memory, compute, result memory, packetizer — are chosen so an array replaces the ALU+FSM block without disturbing the network, credit, or observation contracts.
- Context switching / multi-function pipelines.
- Cell-initiated grant requests.
- Wider-than-32-bit lines (element widths stay bf16-only regardless; §4).
- Loading `A`–`D` from the input memory (v1 writes them via `cfg`/AXIL).
- Cell-destined result sending (`DEST` = another cell) pending the tile-spec cell→cell grant extension (§6.3).
- The tetrad-level control core and the gatling/tetrad engine (§10).

## 4.12 12. Open questions

- Input memory and result memory depths (lines, and flit-conversion for the advertisement) — sized in the MAS against the MAS injection-FIFO and credit-flow analysis.
- `cfg` read reply path: whether rapids's sink half strips headers (TILE_SPEC §11.7 open item) affects nothing here — the cell always builds a full reply header.
- Flush conditions for the packetizer beyond batch-X (idle timeout? end-of-job marker from the FSM?).
- Cell→cell grant/advertisement rules: who grants sink space for cell-destined results, and how a cell advertises its input-memory credits to peer cells (today: to rapids only, §11.10). Needed for `DEST` overrides (§6.3); lands in the tile spec first.
- `DEST` coordinate width tracks the §11.3 `COORD_W` parameterization (4 today); the register is 32-bit with the used low bits defined above.

## 4.13 References

- `alu-infer/PRD.md` — the ALU block's product definition (normative for the ALU since 2026-10-17).
- `tile/docs/TILE_SPEC.md` — adopted tile hierarchy, §11 packet header, credits/grants (normative for §§3, 5, 6).
- `delta_has/ch07_mesh_network/` — the node the cell instantiates.
- `riscv-ip/gatling-rv32itetrad/PRD.md` — owner-authored barrel-core tetrad-engine PRD (leading candidate; deferred per §10).
- `axon_spec.md` — superseded 2026-10-16 (decision trail).
- `2026-10-13-delta-mesh-design.md` — working design record.

---

**Last Updated:** 2026-10-19 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 5 alu-infer — Cell Inference ALU (definition)

The cell's compute ALU: a bf16 inference datapath — two bf16 lanes per 32-bit line — built mostly on the repo's `math_bf16_*` library (adder, multiplier, fused FMA-mac, comparator, min/max, ReLU, and the activation growth set), issued by the tetrad's gatling core. bf16, int8, fp32 and fp8 e4m3 element formats; reduce, PWL LUT activation, const, library activations and conversions alongside the v1 rows.

This directory holds the **product definition and the RTL**; the ALU is integrated and verified at system level inside the cell component (owner decision 2026-10-17; RTL moved here from `cell/rtl/` on 2026-10-20).

- **PRD:** PRD.md
- **MAS:** Micro-Architecture Specification (PDF) — module tree, function rows, fp32 accumulator rows, PWL LUT, conversions, the issue seam, cycle behavior, verification hooks
- **RTL:** `rtl/` — `alu_infer_top.sv` and its module tree (decode, per-format row families, activations, PWL LUT, conversions, accumulator, tag FIFO; package in `rtl/includes/`), with per-block filelists under `rtl/filelists/`, an `alu-infer_all.f` lint master and a four-line area Makefile
- **PWL table builder:** `bin/alu_infer_pwl.py` — fits the 256-entry LUT for sigmoid/tanh/gelu/silu/exp and models the row
- **Unit tests:** `dv/tests/` — `test_alu_infer.py`, run via `make -C dv/tests run-all-<gate|func|full>` after `make clean-all`
- **Issuer:** `gatling-rv32itetrad` — the tetrad barrel core whose programs issue the ALU
- **Integration home:** `cell/`; the cell filelists include this block's filelist.
- **Arithmetic library:** `rtl/math/` — the `math_bf16_*`, IEEE fp32, `math_fp8_e4m3_*` and converter blocks
- **Back to family:** compute-eng-ip README

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 6 rtl-math module index

**RTL:** `rtl/math/` (174 modules) **Tests:** `val/math/` **Common building blocks:** rtl-common

The complete `math_*` arithmetic library: adders, subtractors, multipliers, compressors, prefix cells, and the BF16 / FP8 / FP16 / FP32 / IEEE-754 floating-point modules — every family, with the files that make it up.

## 6.1 Overview

This page is the catalogue, not the guide. Read the **Math Library overview** first: it organizes the whole `math_*` family by operation → methodology — Brent-Kung / Han-Carlson adders, Dadda / Wallace multipliers, IEEE-754 fp32/fp16/bf16/fp8 operators, conversions, and activations — each with its research reference and a link to the detailed doc below, plus the `bin/` code-generation automation. Come back here when you need the exact file list. That's the one thing this page does better than any other.

## 6.2 Module categories

### 6.2.1 Basic arithmetic

- **math_adder_basic** — single-bit adders (full and half adder)
  - Includes: `math_adder_full.sv` (dedicated page), `math_adder_half.sv`, `math_adder_full_nbit.sv`
- **math_adder_ripple_carry** — multi-bit ripple carry adder
- **math_adder_pg_chain** — fast carry lookahead adder
- **math_adder_carry_save** — carry-save adder for multiple operands
  - Includes: `math_adder_carry_save.sv`, `math_adder_carry_save_nbit.sv`

### 6.2.2 Advanced adders

- **math_adder_brent_kung** — Brent-Kung parallel prefix adder family (8/16/32/64-bit)
  - Includes: `math_adder_brent_kung_008.sv`, `math_adder_brent_kung_016.sv`, `math_adder_brent_kung_032.sv`, `math_adder_brent_kung_064.sv`
  - Sub-modules: `math_adder_brent_kung_pg.sv`, `math_adder_brent_kung_black.sv`, `math_adder_brent_kung_gray.sv`, `math_adder_brent_kung_bitwisepg.sv`, `math_adder_brent_kung_grouppg_*.sv`, `math_adder_brent_kung_sum.sv`
- **math_adder_han_carlson** — Han-Carlson hybrid parallel prefix adder family (16/22/32/44/48/72-bit)
  - Includes: `math_adder_han_carlson_016.sv`, `math_adder_han_carlson_022.sv`, `math_adder_han_carlson_032.sv`, `math_adder_han_carlson_044.sv`, `math_adder_han_carlson_048.sv`, `math_adder_han_carlson_072.sv`
  - Building blocks: `math_prefix_cell.sv`, `math_prefix_cell_gray.sv`
- **math_adder_hybrid_ripple_prefix** — hybrid adder: low-order ripple chain + single Kogge-Stone tree for the upper bits (8/16/32/64-bit)
  - Includes: `math_adder_hybrid_ripple_prefix.sv` (parameterized core), `math_adder_hybrid_ripple_prefix_008.sv`, `math_adder_hybrid_ripple_prefix_016.sv`, `math_adder_hybrid_ripple_prefix_032.sv`, `math_adder_hybrid_ripple_prefix_064.sv`

- **math_addsub** — combined adder/subtractor
  - Includes: `math_addsub_full_nbit.sv`

### 6.2.3 Division

- **math_dividers** — integer divider family: restoring, non-restoring, and SRT radix-2 / radix-2-carry-save / radix-4 (parameterized `DATA_WIDTH`)
  - Includes: `math_divider_restoring.sv`, `math_divider_nonrestoring.sv`, `math_divider_srt_radix2.sv`, `math_divider_srt_radix2_csa.sv`, `math_divider_srt_radix2_csa_structural.sv` (cell-built A/B twin), `math_divider_srt_radix4.sv`

### 6.2.4 Subtraction

- **math_subtractor** — subtractor family (single-bit and multi-bit)
  - Includes: `math_subtractor_full.sv`, `math_subtractor_half.sv`, `math_subtractor_full_nbit.sv`, `math_subtractor_ripple_carry.sv`, `math_subtractor_carry_lookahead.sv`

### 6.2.5 Multiplication

- **math_multiplier_wallace_tree** — Wallace tree multiplier family (8/16/32-bit)
  - Includes: `math_multiplier_wallace_tree_008.sv`, `math_multiplier_wallace_tree_016.sv`, `math_multiplier_wallace_tree_032.sv`
  - CSA variants: `math_multiplier_wallace_tree_csa_008.sv`, `math_multiplier_wallace_tree_csa_016.sv`, `math_multiplier_wallace_tree_csa_032.sv`
- **math_multiplier_dadda_tree** — Dadda tree multiplier family (8/16/32-bit)
  - Includes: `math_multiplier_dadda_tree_008.sv`, `math_multiplier_dadda_tree_016.sv`, `math_multiplier_dadda_tree_032.sv`
- **math_multiplier_dadda_4to2** — Dadda tree multiplier with 4:2 compressors (8/11/24-bit)
  - Includes: `math_multiplier_dadda_4to2_008.sv` (BF16), `math_multiplier_dadda_4to2_011.sv` (FP16), `math_multiplier_dadda_4to2_024.sv` (FP32)
  - Building blocks: `math_compressor_4to2.sv`

- **math_multiplier_basic** — basic multiplier components
  - Includes: `math_multiplier_basic_cell.sv`, `math_multiplier_carry_save.sv`

### 6.2.6 BF16 floating-point arithmetic

- **math_bf16_adder** — pipelined BF16 adder with configurable latency
  - Includes: `math_bf16_adder.sv`
  - Dependencies: `shifter_barrel.sv`, `count_leading_zeros.sv`
- **math_bf16_multiplier** — complete BF16 multiplier with IEEE 754 compliance
  - Includes: `math_bf16_multiplier.sv`
  - Sub-modules: `math_bf16_mantissa_mult.sv`, `math_bf16_exponent_adder.sv`
- **math_bf16_mantissa_mult** — BF16 mantissa multiplier with normalization detection
  - Includes: `math_bf16_mantissa_mult.sv`
- **math_bf16_exponent_adder** — BF16 exponent computation with overflow/underflow detection
  - Includes: `math_bf16_exponent_adder.sv`
- **math_bf16_fma** — BF16 Fused Multiply-Add with FP32 accumulator for AI training
  - Includes: `math_bf16_fma.sv`
- **math_bf16_extended** — extended BF16 modules (29 modules)
  - Activation functions: `relu`, `leaky_relu`, `gelu`, `sigmoid`, `tanh`, `silu`, `softmax_8`
  - Math operations: `exp2`, `log2`, `log2_scale`, `reciprocal`, `fast_reciprocal`, `divider`, `goldschmidt_div`, `newton_raphson_recip`
  - Comparison/Selection: `comparator`, `clamp`, `min`, `max`, `min_tree_8`, `max_tree`, `max_tree_8`
  - Format conversions: `to_fp16`, `to_fp32`, `to_fp8_e4m3`, `to_fp8_e5m2`, `to_int`, `int_to_bf16`, `scale_to_int8`

### 6.2.7 FP16 (half-precision) floating-point

- **math_fp16_modules** — FP16 module collection (17 modules)
  - Activation functions: `relu`, `leaky_relu`, `gelu`, `sigmoid`, `tanh`, `silu`, `softmax_8`

- Comparison/Selection: `comparator`, `clamp`, `min`, `max`, `min_tree_8`, `max_tree_8`
- Format conversions: `to_bf16`, `to_fp32`, `to_fp8_e4m3`, `to_fp8_e5m2`

#### 6.2.8 FP32 (single-precision) floating-point

- **math_fp32_modules** — FP32 module collection (17 modules)
  - Activation functions: `relu`, `leaky_relu`, `gelu`, `sigmoid`, `tanh`, `silu`, `softmax_8`
  - Comparison/Selection: `comparator`, `clamp`, `min`, `max`, `min_tree_8`, `max_tree_8`
  - Format conversions: `to_bf16`, `to_fp16`, `to_fp8_e4m3`, `to_fp8_e5m2`

#### 6.2.9 FP8 (8-bit floating-point)

- **math_fp8_modules** — FP8 E4M3 and E5M2 module collection (44 modules)
  - E4M3 (inference): `adder`, `multiplier`, `fma`, activations, comparisons, conversions
  - E5M2 (training): `adder`, `multiplier`, `fma`, activations, comparisons, conversions
  - Format conversions between E4M3, E5M2, BF16, FP16, FP32

#### 6.2.10 IEEE 754-2008 compliant arithmetic

- **math_ieee754_modules** — IEEE 754-2008 arithmetic (12 modules; FTZ by default, full gradual underflow via `SUBNORMAL_SUPPORT=1` on every block; all eight arithmetic units run exact-integer-oracle suites at both parameter values, and the divider/sqrt TBs pin the FSM latencies exactly)
  - FP16: `adder`, `multiplier`, `fma`, `mantissa_mult`, `exponent_adder`
  - FP32: `adder`, `multiplier`, `fma`, `divider`, `sqrt`, `mantissa_mult`, `exponent_adder`
  - Features: `SUBNORMAL_SUPPORT=0` FTZ (legacy default) / `=1` IEEE 754-2008 gradual underflow, complete special-case handling, full status flags

#### 6.2.11 Compressors and prefix cells

- **math_compressor_4to2** — 4:2 compressor for fast parallel reduction
- **math_mod_3_compress** — combinational `X - (X mod 3)` carry-save rounding (monbus record packing)
- **math_prefix_cell** — black cell for parallel prefix adders
  - Includes: `math_prefix_cell.sv`

- **math_prefix_cell_gray** — gray cell for parallel prefix adders (area-optimized)
  - Includes: `math_prefix_cell_gray.sv`

## 6.3 Related Modules

- rtl-common index — counters, FIFOs, arbiters, CDC, data integrity

## 6.4 Navigation

- Documentation index

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 7 Document Information

## 7.1 Overview

### 7.1.1 ALU-INFER Micro-Architecture Specification

*Document information*

| Property | Value |
|---|---|
| Document Title | ALU-INFER Micro-Architecture Specification |
| Version | 0.50 (pinned per owner, “for now”) |
| Date | 2026-10-20 |
| Status | Draft, reconciled to the as-built RTL in `alu-infer/rtl/`. Every row of the PRD’s v1 table, function-set roadmap and growth list is implemented and verified, except what the PRD rules out (a true divider; the integer B4–B32 companion mode; the CGRA replacement). Open decisions are carried as M1–M8 (Chapter 6). |
| Classification | Open Source - MIT License |

### 7.1.2 Document Purpose

This Micro-Architecture Specification (MAS) documents how the cell’s inference ALU is built:

- The `alu_infer_top` module tree and its parameters
- Every function row: the `rtl/math` block it instantiates, its lanes per element format, and its wiring decisions
- The accumulator rows (MAC, REDUCE), the block’s main state
- Decode and the four natural-latency result ports
- The PWL lookup-table activation and the format conversions
- The issue seam between the gatling core and the ALU
- Cycle-level behavior per opcode class, with timing diagrams
- Verification hooks: the library cross-check, the test scenarios and the build configurations

- The configuration surface: parameters, encodings, open items

For what the ALU must do and why, see the **alu-infer PRD** (normative for the ALU block). For the issuer, see the **gatling PRD**. For the cell around both, see the **cell PRD**.

### 7.1.3 Intended Audience

- RTL designers integrating the ALU into the cell
- Verification engineers extending its testbench
- Firmware authors writing gatling programs that issue it
- Students reading an inference datapath top to bottom

## 7.2 Design Notes

### 7.2.1 Revision History

*Revision history*

| Version | Date | Author | Description |
|---|---|---|---|
| 0.50 | 2026-10-20 | RTL Design Sherpa | Roadmap and growth rows implemented and verified: element formats on `func_sel[8:7]` (int8, fp32, fp8 e4m3), REDUCE sum/max with cross-lane fold, PWL LUT, const, library activations, conversions, MAC in every format. Four natural-latency result ports; `MUL_LATENCY`, `MAC_LATENCY`, `EN_*` parameters; `err_opcode`. Issuer is gatling throughout; the execution-FSM framing is retired. Library bug math BUG-009 (exp2) found and fixed. |
| 0.50 | 2026-10-18 | RTL Design Sherpa | Revs pinned at 0.50 per owner. Tag-based elastic |

| Version | Date | Author | Description |
|---|---|---|---|
| | | | alignment (implemented 2026-10-19): `alu_infer_pipe` is a tag FIFO with an arrival join; `PIPE_DEPTH` and `ADDER_LATENCY` replace `ROW_LATENCY`. |
| 0.5 | 2026-10-17 | RTL Design Sherpa | Operand ports stay `LINE_W` each (the v0.7 widening was a line/lane mis-count). |
| 0.4 | 2026-10-17 | RTL Design Sherpa | M7 and the `acc_ctrl` encoding accepted; per-row latency deferred to FPGA bring-up (M8); result flags packed low. |
| 0.3 | 2026-10-17 | RTL Design Sherpa | Same-clock operand capture; scheduler-paced feedback; CLEAR and errored opcodes retire their slots; M5 closed. |
| 0.2 | 2026-10-17 | RTL Design Sherpa | Elastic valid/ready seam at both ends. |
| 0.1 | 2026-10-17 | RTL Design Sherpa | Initial MAS: bf16 lanes on the |

| Version | Date | Author | Description |
|---|---|---|---|
| | | | `math_bf16_*` library, fp32 MAC accumulation (M7). |

## 7.3 References

### 7.3.1 Related Documents

- **alu-infer PRD** (../../../PRD.md) — product definition, normative for the ALU block
- **gatling PRD** (../../../../../riscv-ip/gatling-rv32itetrad/PRD.md) — the barrel RV32I core that issues the ALU
- **cell PRD** (../../../../cell/docs/PRD.md) — the cell around the ALU
- **rtl-math book** (../../../../../../../docs/markdown/rtl-math/index.md) — the library the rows instantiate
- **rtl-math references index** (../../../../../../../rtl/math/references/README.md) — bf16, activation and quantization papers

---

**Last Updated:** 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 8 Design Philosophy and Datapath Organization

## 8.1 What This Block Is

`alu_infer` is the cell's compute engine: a function table over packed element lanes. It is deliberately *not* a processor — no fetch, no program counter, no branches. Programmability lives in the issuer: the tetrad's **gatling** core, a barrel RV32I with the Xcell extension, runs one context per cell. A context's program presents operands and a function select to this block's issue seam and takes results back in order (gatling PRD §1, owner ruling 2026-10-19: *the program is the schedule*). The ALU turns operand lines into result lines, one invocation at a time, and makes no pacing decisions of its own.

Three commitments shape every decision in this specification.

**1. The math library is the datapath.** Every floating-point row instantiates an existing `rtl/math` block — `math_bf16_*` for bf16, the IEEE 754-2008 fp32 adder/multiplier/FMA and `math_fp32_*` for fp32, `math_fp8_e4m3_*` for fp8, the `*_to_*` converters. The ALU inherits each block's structure, numerics and special-value behavior. The int8 rows are plain saturating integer logic; there is no int8 library family.

**2. FMA-first, one-cycle loop.** Inference time is multiply-accumulate time. The accumulator is a per-lane 32-bit bank (fp32 for the float formats, int32 for int8). The bf16 FMA computes the product exactly in fp32 and rounds once, and the accumulate loop is that one fused operation per cycle, so a dependent chain sustains one step per clock at any operand pipeline depth.

**3. A function table, not a function zoo.** `func_sel` names an element format, a row and a sub-op. Decode routes the invocation to one row and to one of four result ports by latency class; nothing arbitrates. New rows are table entries behind the same seam.

## 8.2 The Dataflow in One Paragraph

Gatling presents an invocation: operand lines A and B (`LINE_W` each, captured in the same clock), a 16-bit immediate, `func_sel`, `acc_ctrl`, and `valid`. On the issue edge (`valid && ready`) the invocation takes the next 8-bit tag. Decode checks that the (opcode, format) pair is legal in this build and routes it. A combinational row lands its result in the tag FIFO at the issue edge. The adders land `ADDER_LATENCY` cycles later, the multipliers `MUL_LATENCY` later, and the accumulator rows `MAC_LATENCY` later. Each carries its tag and joins its FIFO entry by tag compare. The FIFO offers results strictly in issue order. An illegal invocation still retires its slot, with a zero line, and pulses `err_opcode`.

### 8.2.1 Figure 1.1: ALU-INFER datapath overview

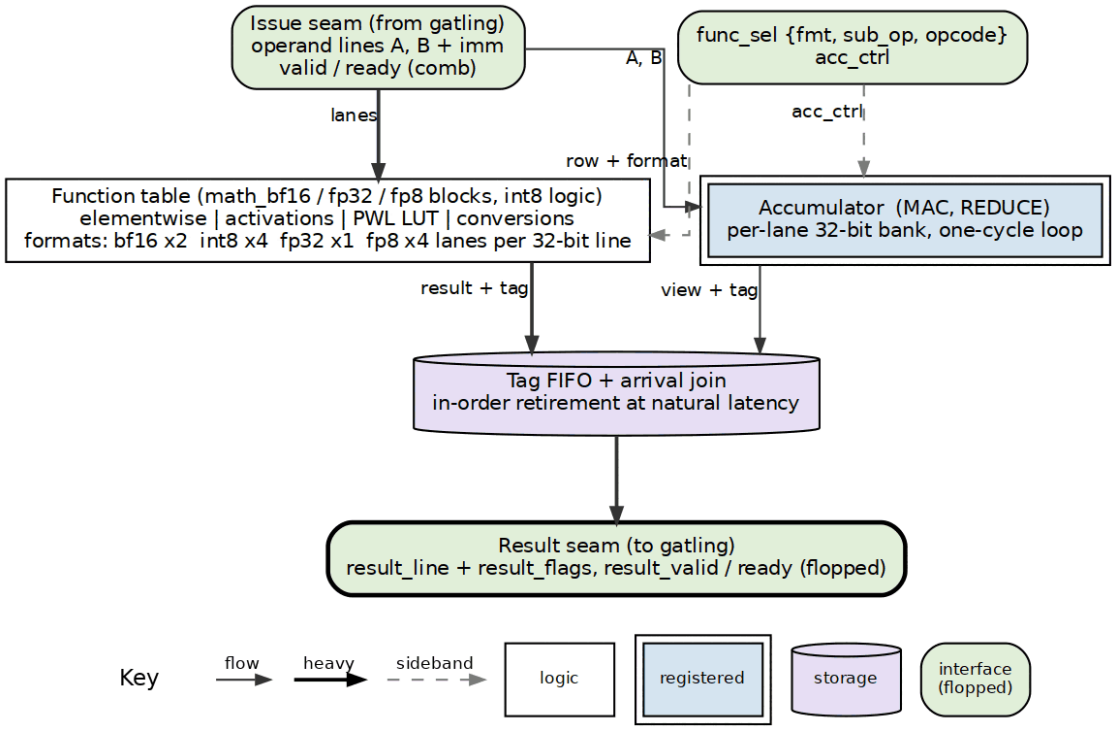



*Figure 1.1: ALU-INFER datapath overview*

The full block diagram, with every row family and result port, is Figure 2.1 in the next chapter.

## 8.3 Element Formats

*Table 1.1: element formats*

| `func_sel[8:7]` | Format | Lanes per 32-bit line | Accumulator lane |
|---|---|---|---|
| 0 | bf16 | 2 | fp32 |
| 1 | int8 (two's complement) | 4 | int32 |
| 2 | fp32 | 1 | fp32 |
| 3 | fp8 e4m3 | 4 | fp32 |

bf16 is the v1 format and the one every row supports. int8 is the PRD roadmap's first added format (4 MACs per clock on a 32-bit line), and fp32 the second. fp8 e4m3 is the §7 element-model extension. Wider lines scale the lane count, never the element width.

## 8.4 Relationship to Gatling and the Cell

The ALU speaks only across the issue seam (Chapter 3) and one status output. Gatling owns sequencing and pacing: it shapes the `valid` stream, obeys `ready`, and pairs results with invocations by counting. Cell configuration — including anything that sets an ALU mode for a layer — arrives in-band on delta, and gatling never originates config traffic (gatling PRD §1). Observability belongs to the Ocelus MonBus plane, and this block exposes no monitor taps.

## 8.5 Status of This Specification

The RTL is built and verified at gate and func levels, in four build configurations (Chapter 5). Chapters 2–4 document the as-built code. Where the PRD left a choice open, this specification records the as-built choice or carries it as M1–M8 (Chapter 6).

---

**Last Updated:** 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 9 alu_infer_top (module tree)

## 9.1 Purpose

`alu_infer_top` is the ALU block the cell instantiates at its issue seam. It owns everything between the operand inputs and the result outputs: decode, the row families, the accumulator, the result ports and the tag FIFO.

## 9.2 Module Tree

```
alu_infer_top                       the ALU block at the issue seam
├── alu_infer_pipe                  tag FIFO + arrival join (one join
per port)
└── alu_infer_rows                  the function table
    ├── alu_infer_decode            legality + result-port routing
    ├── alu_infer_bf16_rows         bf16: add/sub, mul, min/max, cmp,
relu, const
    ├── alu_infer_int8_rows         int8: saturating add/sub/mul,
min/max, cmp, relu, const   [EN_INT8]
    ├── alu_infer_fp32_rows         fp32: IEEE add/sub, mul, min/max,
cmp, relu, const          [EN_FP32]
    ├── alu_infer_fp8_rows          fp8 e4m3: add/sub, mul, min/max,
cmp, relu, const           [EN_FP8]
    ├── alu_infer_act_rows          activations (bf16/fp32/fp8) +
recip/exp2/log2/qscale (bf16) [EN_ACT]
    ├── alu_infer_pwl               256-entry PWL LUT activation
[EN_PWL]
    ├── alu_infer_cvt               CVT and ALTFMT conversions
[EN_CVT]
    ├── alu_infer_delay             tag/data delay lines (adder tag,
MUL_LATENCY)
    └── alu_infer_acc               MAC / REDUCE: operand pipeline +
accumulate stage + bank
        └── alu_infer_delay         MAC_LATENCY operand stages
```

`alu_infer_pkg` (`rtl/includes/`) holds the shared encodings: opcodes, formats, `acc_ctrl`, port numbers. Each `[EN_*]` family is a generate branch: built out, it costs nothing and decodes as illegal (Chapter 2.4). The four planned submodules of the first draft (decompose, select, compose, decode) are absorbed: lane split and pack are part-selects inside each row family, and the select is the per-port mux in `alu_infer_rows`.

### 9.2.1 Figure 2.1: ALU-INFER full block diagram

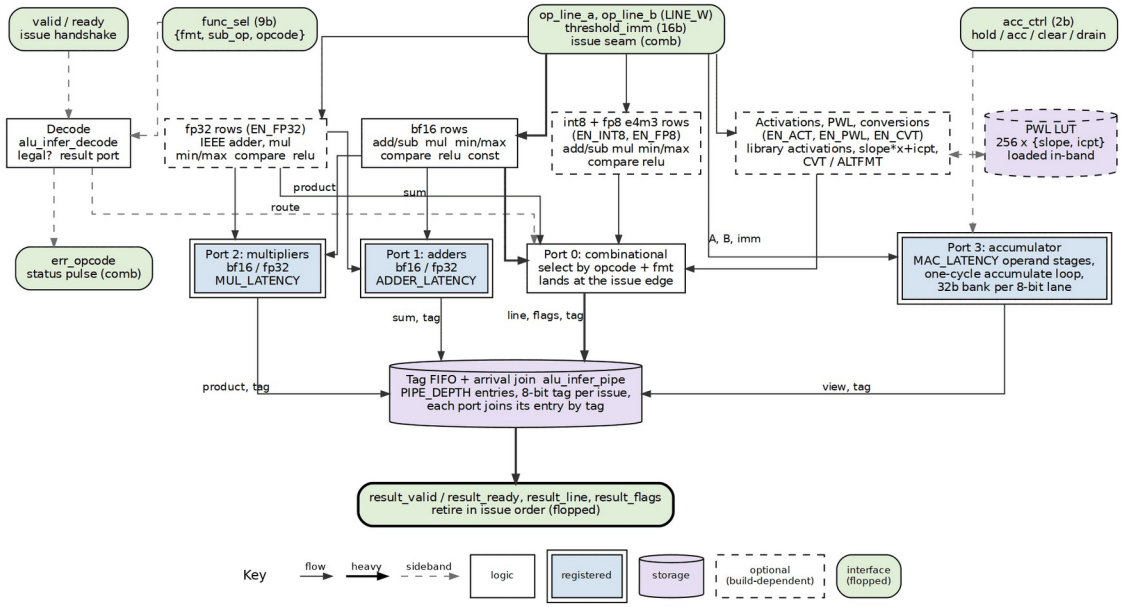



*Figure 2.1: ALU-INFER full block diagram*

The element key is the house v3 key: green rounded boxes are the seam, white boxes combinational logic, double-bordered blue boxes registered, lavender cylinders storage, and dashed outlines build-optional families. The four result ports sit in one rank above the tag FIFO. Each port carries its rows' results and tags at that port's latency.

## 9.3 Parameters

*Table 2.1: `alu_infer_top` parameters*

| Parameter | Default | Meaning |
|---|---|---|
| `LINE_W` | 32 | Line width; a positive multiple of 32. Lanes scale with it, element widths never do |
| `PIPE_DEPTH` | 8 | Tag-FIFO entries = invocations in flight, [2,16]; the stall tolerance, not a latency |
| `ADDER_LATENCY` | 3 | bf16 and fp32 adder pipe stages (the library `PIPE_STAGE_1..4`), [0,4] |
| `MUL_LATENCY` | 0 | bf16 and fp32 multiplier output stages, [0,4]; retiming candidates |
| `MAC_LATENCY` | 0 | Operand stages ahead of the one-cycle accumulate stage, [0,4] |
| `TAG_W` | 8 | Alignment tag width (owner ruling 2026-10-18) |
| `LEAKY_SHIFT` | 4 | Leaky-relu alpha = 2^-LEAKY_SHIFT, every format |
| `EN_INT8`, `EN_FP32`, `EN_FP8` | 1 | Build the element format |
| `EN_REDUCE`, `EN_PWL`, `EN_ACT`, `EN_CVT` | 1 | Build the row family |
| `N_LANES`, `ACC_W` | `LINE_W/16`, 32 | Retained for compatibility; not used by the rows |

## 9.4 Clock and Reset

Single clock, active-low asynchronous `aresetn` (house `ALWAYS_FF_RST`), matching the cell. Reset clears the tag FIFO, the delay lines and the accumulator bank. The PWL table is a memory and has no reset (house SRAM rule): it must be loaded before use (Chapter 2.5).

---

**Last Updated:** 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 10 The Function Rows

Each row is one table entry. It takes the lanes of operand A (and B or the immediate), computes every lane in parallel and independently, and returns one result line. The `rtl/math` block named for a row is its implementation. That block's structure, numerics and special-value handling are documented in its rtl-math page and not restated here; this chapter pins the row's wiring.

## 10.1 Opcode Map

*Table 2.2: the function rows*

| Opcode | Row | Formats | Operands | Port |
|---|---|---|---|---|
| 0 | copy | all | A | comb |
| 1 | add | all | A, B | adder (bf16, fp32) / comb (int8, fp8) |
| 2 | sub | all | A, B | as add |
| 3 | mul | all | A, B | multiplier (bf16, fp32) / comb (int8, fp8) |
| 4 | mac | all | A, B, bank | accumulator |
| 5 | min / max (sub_op[0]) | all | A, B | comb |
| 6 | compare (sub_op[0]: B / imm) | all | A, B or imm | comb |
| 7 | relu | all | A | comb |
| 8 | reduce (sub_op[0]: sum / max; sub_op[1]: fold) | all | A, bank | accumulator |
| 9 | pwl | bf16 | A, LUT | comb |
| 10 | pwl_lut (sub_op[0]: write / read) | bf16 | A = entry, B[7:0] = index | comb |
| 11 | const | all | imm | comb |
| 12 | leaky-relu | bf16, fp32, fp8 | A | comb |
| 13 | gelu | bf16, fp32, fp8 | A | comb |
| 14 | sigmoid | bf16, fp32, fp8 | A | comb |
| 15 | silu | bf16, fp32, fp8 | A | comb |

| Opcode | Row | Formats | Operands | Port |
|---|---|---|---|---|
| 16 | tanh | bf16, fp32, fp8 | A | comb |
| 17 | reciprocal | bf16 | A | comb |
| 18 | exp2 | bf16 | A | comb |
| 19 | log2 | bf16 | A | comb |
| 20 | log2-scale (qscale) | bf16 | A | comb |
| 21 | cvt (src = fmt, dst = sub_op) | library pairs | A (+ B) | comb |
| 22 | altfmt (bf16 <-> fp16 / e5m2) | bf16 | A (+ B) | comb |
| 23–31 | — | — | — | illegal |

“All” means every format built in. A (format, opcode) pair outside this table is illegal (Chapter 2.4).

## 10.2 Elementwise Rows per Format

*Table 2.3: elementwise rows*

| Row | bf16 | int8 | fp32 | fp8 e4m3 |
|---|---|---|---|---|
| add / sub | `math_bf16_adder` (sub: B sign flipped) | saturating `a ± b` | `math_ieee754_2008_fp32_adder` | `math_fp8_e4m3_adder` |
| mul | `math_bf16_multiplier` | saturating `a × b` | `math_ieee754_2008_fp32_multiplier` | `math_fp8_e4m3_multiplier` |
| min / max | `math_bf16_min` / `_max` | signed compare | `math_fp32_min` / `_max` | `math_fp8_e4m3_min` / `_max` |
| compare | `math_bf16_comparator` (lt) | signed `a < b` | `math_fp32_comparator` | `math_fp8_e4m3_comparator` |
| relu | `math_bf16_relu` | `a < 0 ? 0 : a` | `math_fp32_relu` | `math_fp8_e4m3_relu` |
| const | `imm` | `imm[7:0]` | `{imm, 16'h0}` | `imm[7:0]` |

## 10.3 Per-Row Notes

**copy (0).** Operand A to the result line, in any format; useful for moving lines and for test patterns.

**add / sub (1, 2).** Sub shares the adder with B's sign bit flipped (bf16, fp32, fp8). The bf16 and fp32 adders are pipelined `ADDER_LATENCY` stages and land on the adder port; the int8 and fp8 adders are combinational. int8 saturates to [-128, 127] — the inference convention — and never wraps.

**mul (3).** The bf16 and fp32 multipliers are combinational blocks followed by `MUL_LATENCY` output stages. Those registers are retiming candidates: the integration flow moves them into the multiplier. int8 saturates its 16-bit product.

**min / max (5).** `sub_op[0]` picks max. NaN semantics are each block's own: the float blocks return the non-NaN operand (IEEE `minNum`/`maxNum`), recorded at M1.

**compare (6).** Lane `i` of the mask is `A[i] < B[i]`, or `A[i] < imm` when `sub_op[0]` is set. The mask is packed low in `result_flags`, one bit per lane of the format (2, 4, 1 or 4 bits), and `result_line` is zero. The immediate is the raw pattern of the format: a bf16 pattern for bf16, widened exactly to fp32 for fp32, and `imm[7:0]` for int8 and fp8 (M4).

**relu (7).** Negative lanes to zero; NaN passes per the block (M1).

**const (11).** The immediate broadcast into every lane of the format (PRD roadmap item 3): a program builds a bias line without memory traffic.

## 10.4 Activation and Scaling Rows

Opcodes 12–16 instantiate each format's library activation: leaky-relu (alpha = 2^-`LEAKY_SHIFT`), gelu, sigmoid, silu, tanh, in bf16, fp32 and fp8. Opcodes 17–20 are bf16 only, because those blocks exist only in bf16: `math_bf16_reciprocal`, `math_bf16_exp2`, `math_bf16_log2` and `math_bf16_log2_scale` (the quantization scale `2^ceil(log2 max)` of a max value, as bf16).

These rows inherit the library's approximations unchanged. The gelu, sigmoid, silu and tanh blocks are coarse piecewise forms (gelu, for example, is a constant over the medium range). **The PWL row (Chapter 2.5) is the accurate activation path.** It is what the PRD's roadmap item 2 asked for, and its error is bounded and measured. There is no divider row: the PRD (roadmap, 2026-10-19) plans none, and exp followed by reciprocal-multiply covers softmax's divide.

## 10.5 What Every Row Shares

- **Lane independence.** Lane `i` of a result never depends on lane `j` (PRD §4 invariant 1). The accumulator rows' fold sub-op is the one deliberate exception (Chapter 2.3).
- **Natural speed, joined by tag.** A row does not pad itself to match slower ones; its port carries the tag and the FIFO keeps order (Chapter 2.4).
- **FTZ.** Every library block runs at its default `SUBNORMAL_SUPPORT = 0`: subnormal inputs flush to zero (M3).

---

**Last Updated:** 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 11 The Accumulator Rows: MAC and REDUCE

## 11.1 Why a Wide Accumulator

`alu_infer_acc` holds the block's main state: a bank of `LINE_W/8` 32-bit lanes (4 at `LINE_W = 32`). The float formats accumulate in fp32 and int8 in int32. A bf16×bf16 product is **exact** in fp32 (8 × 8 significand bits fit in 24), so `math_bf16_fma` accumulates with one rounding per step and the bank rounds to the element format only when a result is presented. This is the production pattern (owner decision 2026-10-17, PRD §3).

## 11.2 Structure

```
issue --> [ MAC_LATENCY operand stages ] --> accumulate stage --> view
--> port 3
          tag, A, B, imm, func_sel, acc_ctrl      |      ^
                                                  v      |
                                                accumulator bank
```

Every accumulator invocation — CLEAR, HOLD and DRAIN included — enters the operand pipeline in issue order and reads the bank at the accumulate stage, never at issue. A dependent chain is therefore correct at any `MAC_LATENCY`, and it sustains one step per clock: the loop is the single accumulate stage, and the operand stages sit ahead of it.

*Table 2.4: accumulate operation per format*

| Format | Lanes used | MAC update | REDUCE sum | REDUCE max |
|---|---|---|---|---|
| bf16 | 0 .. LINE_W/16-1 | `math_bf16_fma(a, b, acc)` | `fma(a, 1.0, acc)` | `math_fp32_max(acc, a)` |
| fp8 e4m3 | 0 .. LINE_W/8-1 | operands widened exactly to bf16, then as bf16 | as bf16 | as bf16 |
| fp32 | 0 .. LINE_W/32-1 | `math_ieee754_2008_fp32_fma(a, b, acc)` | `fma(a, 1.0, acc)` | `math_fp32_max(acc, a)` |
| int8 | 0 .. LINE_W/8-1 | `acc + a*b` (int32, wraps) | `acc + a` | `max(acc, a)` |

The bank is shared across formats. Lanes a format does not use hold their value; a program that changes format mid-layer issues CLEAR first.

## 11.3 Why the Loop Stays Fused

The draft MAS described a split row: an exact-product stage, pipelined, ahead of a one-cycle fp32 add in the loop. As built, the loop is the fused library block. A 4-million-vector equivalence check of `fp32_adder(exact_product, acc)` against `math_bf16_fma(a, b, acc)` found 24,987 differences. The two classes were a product that overflows fp32 against an infinite accumulator (the fused block returns the accumulator, the split returns NaN), and subnormal accumulator pass-through. The split would trade the library's numerics for a loop the 8×8 bf16 product barely lengthens. Depth goes into the operand stages instead, where the integration flow can retime it into the product half.

## 11.4 acc_ctrl

*Table 2.5: accumulator actions*

| acc_ctrl | Action |
|---|---|
| HOLD (00) | bank unchanged; present it |
| ACCUMULATE (01) | bank ← update (Table 2.4) per active lane; present the new bank |
| CLEAR (10) | bank ← the operation's identity: +0 for MAC and sum, −inf for max (int8: INT32_MIN) |
| DRAIN (11) | bank unchanged; present it (the layer's final read-out) |

## 11.5 Presentation (decision M7)

Every accumulator invocation retires one result line: the post-update bank rounded to the element format. That is `math_fp32_to_bf16` for bf16 and `math_fp32_to_fp8_e4m3` for fp8. fp32 presents the bank exactly. int8 presents `sat8(round(acc / 2^imm[4:0]))`, the requantize shift (round half up), so an int8 layer drains to int8 without a separate pass. The bank itself never rounds: the views are lossy, the state is not.

## 11.6 REDUCE and the Cross-Lane Fold

REDUCE folds a span of lines into the bank. Each ACCUMULATE adds line A (sum) or keeps its maximum (max), lane by lane; B is unused. CLEAR starts a span and DRAIN reads it out. This is the PRD's roadmap item 1, which unlocks softmax, layernorm and pooling.

A per-lane result still leaves `N` partial results. **`sub_op[1]` presents the cross-lane fold**: the sum (or max) of the active lanes, in lane order (`((l0 + l1) + l2) + l3`, fp32 adds), broadcast to every lane of the result. The bank is unchanged, so a program can DRAIN the per-lane values and the fold from the same span. Softmax uses it directly: REDUCE max with fold gives the row maximum in every lane, ready for the subtract.

## 11.7 Isolation Invariants

- The bank is read and written only by the accumulator rows (PRD §4 invariant 2, extended from MAC to REDUCE).
- CLEAR or reset is the only way to re-initialize it, so a chain is reproducible from `{reset, acc_ctrl sequence, operand sequence}`.
- Outside the fold, lane `i` of the bank never sees lane `j`'s operands.

---

**Last Updated:** 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 12 Decode and Result Ports

## 12.1 Decode (`alu_infer_decode`)

A combinational check of `func_sel` against Table 2.2 and the build's `EN_*` parameters. It returns `legal` and the result port. An invocation is **illegal** when its opcode is unassigned (23–31), its format is not defined for the row (an int8 activation, a PWL in fp32, reciprocal in fp8), its conversion pair has no library converter, or its format or family is built out.

An illegal invocation is not dropped. It routes to the combinational port with a zero line and zero flags and **retires its slot in order** (owner ruling 2026-10-17): the pipeline never deadlocks and gatling's count pairing never breaks. On its issue edge `err_opcode` pulses for one cycle. That is a status output for the cell's error IRQ, not seam flow control (PRD §4: the seam carries no error signal).

## 12.2 The Four Result Ports

*Table 2.6: result ports*

| Port | Rows | Latency after the issue edge | Implementation |
|---|---|---|---|
| 0 combinational | copy, min/max, compare, relu, const, int8 and fp8 arithmetic, fp32 non-arithmetic, activations, PWL, conversions, illegal | 0 (written at the issue edge) | per-format result mux in `alu_infer_rows` |
| 1 adder | bf16 and fp32 add/sub | `ADDER_LATENCY` | library adder pipes; the tag and format ride an `alu_infer_delay` alongside |
| 2 multiplier | bf16 and fp32 mul | `MUL_LATENCY` | combinational multiplier, then `alu_infer_delay` of {tag, product} |
| 3 accumulator | MAC, REDUCE | `MAC_LATENCY` | `alu_infer_acc` (Chapter 2.3) |

Each port carries `{valid, tag, line, flags}`. Within a port, results are in issue order. Across ports, several may land in the same cycle, each in its own FIFO entry.

## 12.3 The Tag FIFO and Arrival Join (`alu_infer_pipe`)

Owner ruling 2026-10-18: each issue takes an 8-bit tag, a FIFO tracks the tags in flight, and a result is offered only when every op of its tag group has arrived. As built:

- A free-running counter assigns the tag at issue, and the FIFO allocates the tail entry `{tag, pending}`.
- Each port that lands searches the entries for its tag (unique while in flight: `PIPE_DEPTH` ≤ 16 against a 256-value space) and fills `{line, flags, arrived}`. A port landing in its own issue cycle (latency
  0) writes the entry being allocated.
- The head is offered when it has arrived. A fast result behind a slow one waits at the head (owner ruling: following ops align with or after the earlier slower operation).
- `ready` is high while the FIFO has a free entry, or when the head retires this cycle. A stalled consumer blocks issue only once `PIPE_DEPTH` invocations are in flight.

**Tag groups.** The seam issues one `func_sel` per invocation, so a tag group is one op and the join waits for one arrival. A multi-op group needs a seam that issues several ops at once, which no document defines yet; the per-port tag compare is the mechanism that group would use.

---

**Last Updated:** 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 13 The PWL LUT Activation

PRD roadmap item 2: a 256-entry lookup table with linear interpolation, covering sigmoid, tanh, gelu, silu and exp with one row and bounded error.

## 13.1 Segments

The segment index of a bf16 x is `{sign, octave[3:0], mant[6:4]}`, where `octave = exp − 120`, clamped to [0, 15]. Segments are log-spaced: |x| in [2^-7, 2^9) at eight segments per octave, both signs.

- |x| < 2^-7 uses segment `{sign, 0, 0}`, whose fit covers [0, 2^-7·1.125).
- |x| ≥ 2^9, inf and NaN use segment `{sign, 15, 7}`, the saturating tail.

## 13.2 Evaluation

An entry is `{slope bf16 [31:16], intercept bf16 [15:0]}`. Each lane computes `bf16(math_bf16_fma(slope, x, {icpt, 16'h0}))`: one fp32 rounding, then RNE to bf16. An infinite `x` against a zero slope returns the intercept — the saturating value — instead of the `0 × inf` NaN. A NaN `x` returns NaN.

## 13.3 Loading In-Band

The table is loaded through the seam itself, like everything else the tile receives:

- `OP_PWL_LUT`, `sub_op = 0`: write entry `B[7:0]` with line A. It retires a zero line.
- `OP_PWL_LUT`, `sub_op = 1`: read entry `B[7:0]` back as the result line.

256 writes stream at one per clock, so a function switch costs 256 cycles. The table is a memory (distributed RAM, no reset); it must be loaded before the first `OP_PWL`.

## 13.4 The Table Builder

`alu-infer/bin/alu_infer_pwl.py` fits each segment by least squares, rounds the slope to bf16, and refits the intercept against the rounded slope (the slope's rounding error dominates otherwise). It writes 256 hex words (`--hex`) and reports the modeled error (`--report`). It also carries the row's bit-level model, which the testbench checks the RTL against.

*Table 2.7: modeled PWL accuracy (relative error only where $|f| > 2^{-6}$)*

| Function | Domain | Max abs error | Max rel error |
|---|---|---|---|
| sigmoid | [−12, 12] | 3.8e-3 | 2.3e-2 |
| tanh | [−12, 12] | 3.3e-3 | 4.0e-3 |
| gelu | [−12, 12] | 8.2e-3 | 1.6e-2 |
| silu | [−12, 12] | 1.6e-2 | 2.1e-2 |
| exp | [−16, 4] | — | 2.9e-2 |

The sigmoid and tanh errors are at bf16's output resolution near 1 ($2^{-8} \approx 3.9e-3$). exp is reported where softmax uses it ($x - \max \le 0$): an octave-wide segment is too coarse for exp at large $|x|$, and exp(−16) is already below anything softmax keeps.

---

**Last Updated:** 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 14 Format Conversions

`OP_CVT` converts from the format in `func_sel[8:7]` to the format in `sub_op`. `OP_ALTFMT` covers the interchange formats that are not ALU element formats. Every conversion is an `rtl/math` converter; this is the PRD's growth item "format conversions" plus the int8 drain.

## 14.1 Lane Packing

- **Narrowing** (narrower destination lanes): line A fills the low destination lanes and line B the next ones. Two source lines pack one result line — bf16 → int8 turns four bf16 values (two lines) into one line of four int8.
- **Widening** (wider destination lanes): one line A yields one part of its lanes. `B[0]` selects the half, or `B[1:0]` the quarter for fp8 → fp32.
- **Same format**: a copy.

## 14.2 Pairs

*Table 2.8: CVT pairs*

| src → dst | Converter | Packing |
|---|---|---|
| bf16 → int8 | `math_bf16_scale_to_int8`, scale = `threshold_imm` (bf16) | narrow ×2 |
| bf16 → fp32 | `math_bf16_to_fp32` | widen, half `B[0]` |
| bf16 → fp8 | `math_bf16_to_fp8_e4m3` | narrow ×2 |
| int8 → bf16 | `math_int_to_bf16` (`INT_WIDTH = 8`), unscaled | widen, half `B[0]` |
| fp32 → bf16 | `math_fp32_to_bf16` | narrow ×2 |
| fp32 → fp8 | `math_fp32_to_fp8_e4m3` | narrow ×2 (two of four lanes filled) |
| fp8 → bf16 | `math_fp8_e4m3_to_bf16` | widen, half `B[0]` |
| fp8 → fp32 | `math_fp8_e4m3_to_fp32` | widen, quarter `B[1:0]` |
| int8 ↔ fp32, int8 ↔ fp8 | none in the library | illegal |

bf16 → int8 is the **int8 drain** (`math_bf16_scale_to_int8`, growth list): it quantizes with the immediate as scale. int8 → bf16 is unscaled; the dequantize scale is a following `OP_MUL`.

## 14.3 ALTFMT (bf16 only)

*Table 2.9: ALTFMT sub-ops*

| sub_op | Conversion | Packing |
|---|---|---|
| 0 | bf16 → fp16 (`math_bf16_to_fp16`) | lane for lane |
| 1 | fp16 → bf16 (`math_fp16_to_bf16`) | lane for lane |
| 2 | bf16 → fp8 e5m2 (`math_bf16_to_fp8_e5m2`) | narrow ×2 |
| 3 | fp8 e5m2 → bf16 (`math_fp8_e5m2_to_bf16`) | widen, half `B[0]` |

**Last Updated:** 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

# 15 The Issue Seam

The seam between the issuer and this block is the ALU's whole external surface, plus one status output. The issuer is the tetrad's **gatling** core (owner ruling 2026-10-19): a context's program presents an invocation's operands and function, and takes the result line back. Which Xcell instructions and cell glue drive these signals is gatling-MAS scope. This chapter fixes what the ALU sees.

## 15.1 Signal Contract

*Table 3.1: issue-seam signals*

| Signal | Dir (ALU view) | Width | Semantics |
|---|---|---|---|
| `op_line_a` | in | `LINE_W` | operand line A; captured at the issue edge together with B (owner ruling 2026-10-17: two ops scheduled in the same clock finish at the same time) |
| `op_line_b` | in | `LINE_W` | operand line B (unused by unary rows; the PWL index, conversion part select) |
| `threshold_imm` | in | 16 | immediate: compare threshold, const value, quantize scale, int8 requantize shift (`[4:0]`) |
| `func_sel` | in | 9 | `{fmt[8:7], sub_op[6:5], opcode[4:0]}` — sampled at issue |
| `acc_ctrl` | in | 2 | accumulator action for MAC and REDUCE (Table 2.5); ignored by other rows |
| `valid` | in | 1 | the issuer presents an invocation; all inputs stable while `valid && !ready` |

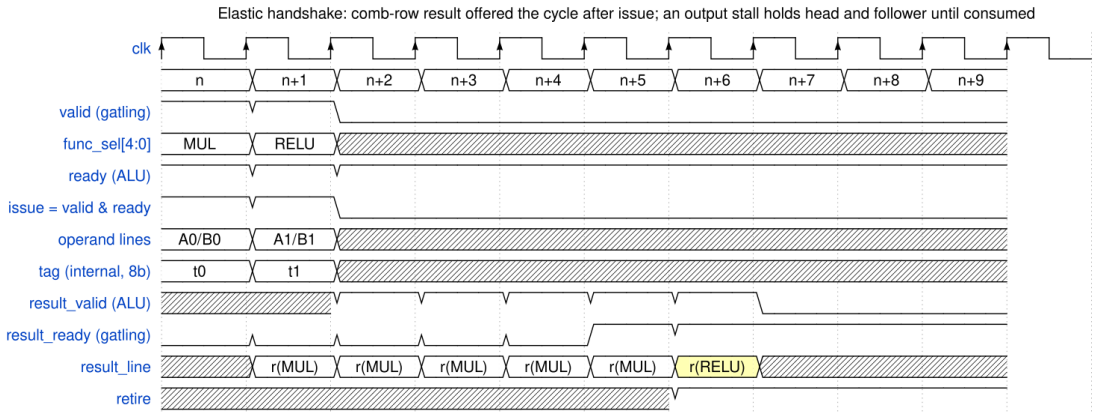
| Signal | Dir (ALU view) | Width | Semantics |
|---|---|---|---|
| `ready` | out | 1 | the tag FIFO has room (or the head retires this cycle) |
| `result_valid` | out | 1 | the head result is offered; held with stable data until consumed |
| `result_ready` | in | 1 | the issuer consumes the offered result |
| `result_line` | out | `LINE_W` | the result, one per invocation, in issue order |
| `result_flags` | out | `LINE_W` | compare mask, one bit per lane packed low (M5); zero otherwise |
| `err_opcode` | out | 1 | pulses on the issue edge of an illegal `func_sel` (status, not flow control) |
| `clk`, `aresetn` | in | 1 | single clock, asynchronous active-low reset |

An **issue** is a rising edge with `valid && ready`; a **retire** is a rising edge with `result_valid && result_ready`.

## 15.2 Handshake Rules

1. **Issue is `valid && ready`.** `ready` is high whenever the FIFO has room, so the issuer may present a new invocation **every clock** (II=1). It falls only once `PIPE_DEPTH` invocations are in flight behind a stalled consumer.
2. **Retire is `result_valid && result_ready`, strictly in issue order.** Each result is offered at its port's natural latency (Table 2.6) and never before every earlier invocation has retired. A stalled consumer holds `result_valid` with stable data; results behind it wait in their own entries.
3. **Elastic end to end.** There is no alignment state to freeze: entries hold their arrivals indefinitely, and when `result_ready` returns, the head retires and the rest drain one per cycle.
4. **In order by construction.** The issuer pairs result N with invocation N by counting. The 8-bit tags are internal and never on the seam (owner ruling 2026-10-18).
5. **Config sampled at issue.** `func_sel`, `acc_ctrl` and the immediate travel with the invocation, so the issuer may change them for the next issue at once.
6. **Every issue retires.** Illegal invocations, CLEAR, and PWL table writes all retire one line (zero for the illegal and the writes).
7. **Scheduler-paced feedback** (owner ruling 2026-10-17). Where results feed back — the accumulator chain, a program re-presenting a result as an operand — the issuer owns the pace. The ALU makes no autonomous pacing decisions. Because the accumulate loop is one cycle, a dependent MAC or REDUCE chain may be issued every clock.

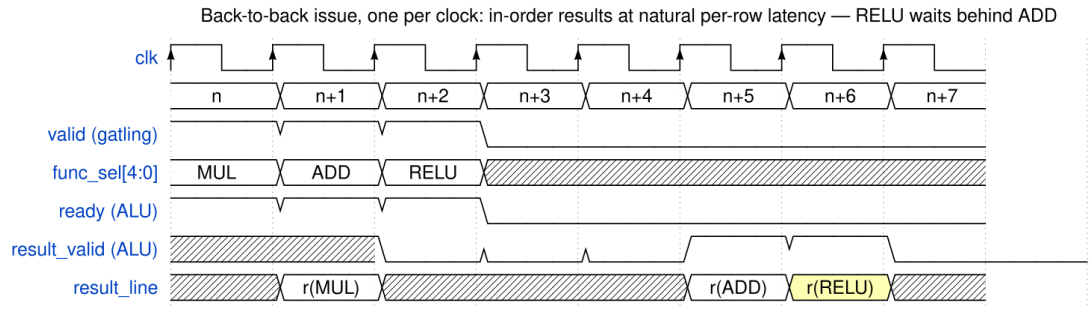
### 15.2.1 Waveform 3.1: Elastic handshake — natural latency, output stall, ready stays high




*Waveform 3.1: elastic handshake*

Two combinational invocations (MUL at `MUL_LATENCY = 0`, then RELU). The MUL issued at n is offered at n+1 and the RELU lands behind it. The consumer stalls from n+2 to n+4: the head holds `r(MUL)` with stable data and **ready stays high**, because a two-entry backup is nowhere near `PIPE_DEPTH`. The consumer accepts at n+5 and n+6.

### 15.2.2 Waveform 3.2: Back-to-back issue — mixed latencies, in-order results




*Waveform 3.2: back-to-back issue*

MUL, ADD and RELU on consecutive clocks with the output draining. The MUL retires at n+1. The head then belongs to the ADD, whose adder port lands at n+5 (`ADDER_LATENCY = 3`); `result_valid` dips while the RELU, computed long before, waits behind it. ADD retires at n+5 and RELU right after. No row carries padded stages.

## 15.3 Boundary Notes

- **Configuration.** Cell configuration arrives in-band on delta and gatling never originates config traffic (gatling PRD §1). The ALU sees configuration only as the per-invocation `func_sel` / `acc_ctrl` / immediate. The PWL table is loaded through the seam (Chapter 2.5).
- **Observability.** Ocelus owns it; the ALU has no monitor taps. The accumulator is readable through DRAIN and the PWL table through `OP_PWL_LUT` read, so no debug port is needed.

---

**Last Updated:** 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 16 The Compute Cycle, per Opcode Class

Every invocation is the same three beats — **present, compute, retire** — but under the elastic handshake (Chapter 3) the beats of consecutive invocations overlap: while one computes, the next is presented and the previous retires. This chapter walks each class as if it were alone; the overlap rules at the end apply to all of them together.

## 16.1 Unary Rows (copy, relu, activations, PWL, const)

The issuer presents line A (const uses only the immediate). Every lane is computed in parallel and the result lands on the combinational port at the issue edge, so it is offered the next cycle.

## 16.2 Binary Rows (add, sub, mul, min/max, compare)

Lines A and B arrive in the same clock (owner ruling 2026-10-17), and the row pairs lane `i` of A with lane `i` of B. bf16 and fp32 add/sub land `ADDER_LATENCY` cycles later, bf16 and fp32 mul `MUL_LATENCY` cycles later, and everything else at the issue edge. Compare packs its mask into `result_flags` and returns a zero `result_line`.

## 16.3 Conversions

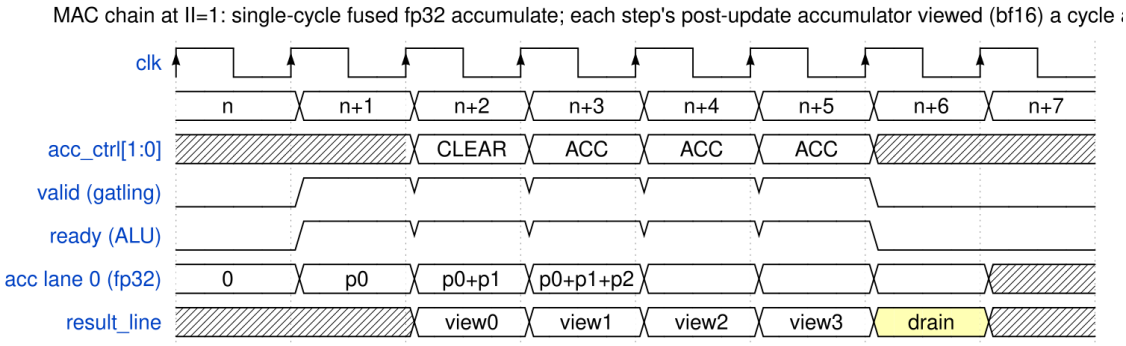
Narrowing conversions consume both lines and pack them into one result; widening conversions consume one line and select the part with `B[1:0]` (Chapter 2.6). All land at the issue edge.

## 16.4 MAC and REDUCE (the chains that matter)

An accumulator invocation enters the operand pipeline and reaches the accumulate stage `MAC_LATENCY` cycles later, where `acc_ctrl` chooses the action:

1. **CLEAR** starts a layer or span: the bank takes the operation's identity (+0, or −inf for max). The invocation flows through like any other and retires the cleared view, so count pairing holds.
2. **ACCUMULATE** per step: MAC adds `a × b` (fused, one rounding), REDUCE adds `a` or keeps the maximum. The result line is the post-update bank in the element format (M7).
3. **DRAIN** reads out the final values without changing the bank. With REDUCE's `sub_op[1]` it presents the cross-lane fold instead.

### 16.4.1 Waveform 4.1: MAC chain — clear, three accumulates, drain




*Waveform 4.1: MAC chain*

At `MAC_LATENCY = 0` each step is offered the cycle after its issue. A deeper `MAC_LATENCY` delays every view by the same amount and keeps one step per clock, because the loop is the accumulate stage alone. The testbench proves both at `MAC_LATENCY` 0 and 2 on a 104-to-512-step chain (Chapter 5).

## 16.5 Overlapping Invocations

The pipeline never inserts a stall of its own. A MUL, a REDUCE and a PWL may issue on consecutive clocks, all in flight together, retiring in issue order as each one's port lands. Three consequences:

- **No drain logic.** Switching opcode or format mid-stream costs nothing: retirement is in issue order regardless of port latency.
- **The issuer pairs results by counting.** No tags on the seam.
- **Idle is a hold.** When `valid` drops, in-flight invocations drain, the last result holds until consumed, and the bank holds.

---

**Last Updated:** 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 17 Verification Hooks and Observability

The tests live in `alu-infer/dv/`. `tbclasses/alu_infer_tb.py` is the testbench (`AluInferTB` on the repo's `TBBase`), with the extended scenarios mixed in from `alu_infer_ext_tests.py`. `alu_infer_golden.py` holds the golden models, and `tests/test_alu_infer.py` is a thin `TEST_TYPE` dispatcher (the stream-area standard pattern).

Both valid/ready interfaces are owned by RDS-DV GAXI BFMs: a `GAXIMaster` drives issue, a `GAXISlave` receives results, and passive `GAXIMonitor`s record both. Timing checks use the BFMs' packet timestamps. The extended scenarios issue invocations as bursts, so the elastic pipeline streams while every retired line is checked.

## 17.1 The Golden Models: the Library's Own

Every floating-point row instantiates an `rtl/math` block, so its expected value comes from **the same reference model that block's unit test uses** (`bin/TBClasses/common/fp_testing.py`, `bf16_testing.py`), at that test's tolerance. Exact rows (copy, const, min/max, relu) must match exactly; arithmetic rows within 1 ULP; reciprocal 2, exp2 and log2 16 (log2 200 near zero), as their library tests allow. This is the MAS's built-in cross-check made literal: an ALU/library divergence is a wiring bug in exactly one of them.

It has already paid off. The exp2 row's check exposed **math BUG-009**: `math_bf16_exp2` returned 0 for every $x \geq 256$, because a 9-bit signed "+256" saturate wrapped to $-256$. It was fixed in `rtl/math`, and the library's own test gained the cases that failed.

- **int8**: the exact saturating model (no library family).
- **Accumulators**: the bank is modeled bit-exactly for bf16, fp8 and int8 (the `math_bf16_fma` contract, including its FTZ of an underflowing product). The fp32 bank is checked against the library FMA reference at its tolerance and then re-synced from the DUT, which presents the fp32 bank exactly.
- **PWL**: bit-exact against the row model in `bin/alu_infer_pwl.py`, plus an approximation bound per function (Table 2.7, with margin).

## 17.2 Scenarios

*Table 5.1: test scenarios*

| TEST_TYPE | Proves |
|---|---|
| copy_timing, tag_join, backpressure | handshake, natural latency, II=1, the tag join, the elastic hold |
| add_sub, mul, minmax, compare, relu, special_grid, sub_ulp_corner | the v1 bf16 rows, special-value grid, the adder's 1-ULP corner |
| mac, mac_chain | bf16 CLEAR/ACC/DRAIN/HOLD; a 104–512-step dependent chain at II=1, every step exact |
| fmt_int8, fmt_fp32, fmt_fp8 | every elementwise row per format, specials and random values |
| const, act, cvt, pwl | const in every format; activations; every CVT pair and ALTFMT; PWL load, read-back, evaluation and accuracy |
| reduce, mac_formats | REDUCE sum/max with identity CLEAR and fold, every format; MAC in int8/fp32/fp8 |
| latency | each port at its parameterized latency, every format that uses it; in-order retirement behind the slowest port |
| error_opcode | every illegal class retires a zero line in order and pulses `err_opcode` exactly once |
| mixed_soak | random mixed opcodes under the GAXI delay profiles |
| softmax | the roadmap rows composed as a program would: REDUCE max + fold, SUB, PWL exp, REDUCE sum + fold, RECIP, MUL, each step fed by the last; within 2e-2 of float softmax (measured 1.6e-3) |

## 17.3 Build Configurations

*Table 5.2: build configurations*

| Config | Parameters | Runs |
|---|---|---|
| default | RTL defaults (`ADDER_LATENCY` 3, `MUL_LATENCY` 0, `MAC_LATENCY` 0) | every scenario at `PIPE_DEPTH` 4 and 8 |
| deep | `ADDER_LATENCY` 4, `MUL_LATENCY` 2, `MAC_LATENCY` 2 | the timing and accumulator scenarios |
| flat | every latency 0 | the timing and accumulator scenarios (exercises the latency-0 tag bypass) |
| lean | every `EN_*` off | the v1 suite, and every built-out family must decode illegal |

The house gate/func/full grid applies: gate runs every cell lightly, and func and full deepen the value sets and chains and sweep the mixed soak across the standard GAXI delay profiles.

## 17.4 Coverage Matrix

*Table 5.3: coverage matrix*

| Axis | Values |
|---|---|
| Opcode × format | every legal pair of Table 2.2; every illegal class |
| Special-value class | ±0, ±inf (fp8: ±max-normal), NaN, ±min-subnormal, ±1, per format (`special_value_grid`) |
| Lane position | every lane of every format |
| Accumulator | CLEAR/ACC/DRAIN/HOLD; chains over 100 steps; REDUCE fold sum and max |
| Latency | 0, the defaults, and deep for each port |

`dv/testplans/alu_infer_top_testplan.yaml` maps each scenario to the RTL lines its own merged coverage data covers (generated, never hand-edited), and `bin/cov_utils/verify_testplan_coverage.py --strict` re-checks it.

## 17.5 Observability

- **No internal monitor taps.** Observability is the Ocelus MonBus plane's job.
- **State is readable through the seam**: the accumulator by DRAIN and the PWL table by `OP_PWL_LUT` read.
- **M2** (whether the library blocks' IEEE exception outputs should reach `result_flags`) stays open; `err_opcode` covers illegal invocations only.

---

**Last Updated:** 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 18 Parameters, Encodings, and Open Items

## 18.1 Parameters

See Table 2.1. Two kinds:

- **Latency knobs** (`ADDER_LATENCY`, `MUL_LATENCY`, `MAC_LATENCY`, `PIPE_DEPTH`): none of them changes behavior seen at the seam, only when results are offered. The tag join absorbs any combination.
- **Build-out switches** (`EN_*`): a family built out decodes as illegal. The lean build (all off) is the v1 bf16 ALU.

## 18.2 `func_sel` Encoding

*Table 6.1: `func_sel` fields*

| Bits | Field | Encoding |
|---|---|---|
| [4:0] | opcode | Table 2.2; 23–31 unassigned (illegal) |
| [6:5] | sub_op | row-specific: min/max select, compare operand, reduce sum/max and fold, PWL write/read, CVT destination format, ALTFMT conversion |
| [8:7] | fmt | element format: 0 bf16, 1 int8, 2 fp32, 3 fp8 e4m3 |

## 18.3 `acc_ctrl` Encoding

*Table 6.2: `acc_ctrl` encoding (accepted 2026-10-17, “works for now”)*

| Value | Name | Action |
|---|---|---|
| 2’b00 | HOLD | bank unchanged; present it |
| 2’b01 | ACCUMULATE | MAC / REDUCE update per active lane |
| 2’b10 | CLEAR | bank ← identity (+0; −inf or INT32_MIN for max) |
| 2’b11 | DRAIN | bank unchanged; present it (or its fold) |

## 18.4 `threshold_imm` Uses

*Table 6.3: immediate uses*

| Row | Meaning |
|---|---|
| compare (`sub_op[0]` = 1) | threshold, as one lane of the format |
| const | the value broadcast |
| int8 MAC / REDUCE | `[4:0]` = requantize right shift of the presented view |
| CVT bf16 → int8 | quantization scale (bf16) |

## 18.5 Filelists

`alu-infer/rtl/filelists/alu_infer_top.f` is the block's compile closure: `alu_infer_pkg`, the `rtl/math` filelists of every instantiated block, and the area's sources. The unit testbench compiles it. `filelists/alu-infer_all.f` is the area lint master, the name `rtl/make/area.mk` derives from `AREA := alu-infer`. The cell filelists include the block's filelist for integration.

## 18.6 Open Items Register

*Table 6.4: open items M1–M8*

| ID | Item | State |
|---|---|---|
| M1 | NaN/inf semantics in min/max and activations | **Recorded as built**: each library block's semantics (min/max return the non-NaN operand; relu and the activations pass NaN). Owner may override. |
| M2 | Whether library IEEE exception outputs forward into `result_flags` | Open (observability) |
| M3 | Denormal/FTZ policy | **Recorded as built**: FTZ everywhere (every block's default `SUBNORMAL_SUPPORT = 0`). Owner may override. |
| M4 | `threshold_imm` encoding | **Recorded as built**: the raw element pattern of the format (Table 6.3) |
| M5 | `result_flags` format | Closed 2026-10-17: one mask bit per lane, packed low |
| M6 | `func_sel[8:7]` disposition | **Closed by implementation 2026-10-20**: the element-format field, as the PRD roadmap planned |
| M7 | MAC presents the post-update bank in the element format; DRAIN presents without updating | Accepted 2026-10-17; extended to REDUCE and every format |
| M8 | Per-row latency values tuned on the FPGA | Open (FPGA bring-up); the latency parameters are the knobs |

**Last Updated:** 2026-10-20